République Tunisienne

Ministère de l'Enseignement Supérieur

et de la Recherche Scientifique

Université de Tunis El Manar

Institut Supérieur d'Informatique d'El Manar

Rapport de Stage de Fin d'Études

Présenté en vue de l'obtention du

Diplôme en Electronique Electrotechnique et Automatisme

Spécialité : Systèmes Embarqués

Par

Dhia SOUDANI et Aymen MANSOURI

ROBOT ASSISTANT HUMANOÏDE

Encadrant académique : **Mohamed GLAOUI** Maître Assistant

Réalisé au sein de *l'Institut Supérieur d'Informatique*

Année Universitaire 2024 - 2025

République Tunisienne

Ministère de l'Enseignement Supérieur

et de la Recherche Scientifique

Université de Tunis El Manar

Institut Supérieur d'Informatique d'El Manar

Rapport de Stage de Fin d'Études

Présenté en vue de l'obtention du

Diplôme en Electronique Electrotechnique et Automatisme

Spécialité : Systèmes Embarqués

Par

Dhia SOUDANI et Aymen MANSOURI

ROBOT ASSISTANT HUMANOÏDE

Encadrant académique : **Mohamed GLAOUI** Maître Assistant

Réalisé au sein de *l'Institut Supérieur d'Informatique*

Année Universitaire 2024 - 2025

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant académique, **Mohamed GLAOUI**

Signature

Dédicaces

Je dédie ce travail à toutes les personnes qui m'ont accompagné et soutenu tout au long de ce parcours.

À mes parents, Kais et Houda,

Pour vos sacrifices immenses, votre amour inconditionnel et votre soutien sans faille qui m'ont porté chaque jour. Votre patience et votre dévouement sont les fondations de tout ce que j'ai accompli.

À ma sœur Zeineb,

Pour ta présence constante, ton écoute, et ta complicité qui m'ont été d'un grand réconfort.

À mes oncles Mohamed, Slim et Kamel, et **à mes tantes Karima et Hanen**,

Merci pour vos encouragements, vos conseils et votre bienveillance.

À Nada, ma complice de toujours,,

Pour ta présence fidèle dans toutes les circonstances, tes encouragements constants, et ton soutien indéfectible depuis l'enfance.

À mon ami Wahid,

Pour ton accompagnement, ta guidance et ta présence précieuse tout au long de ce chemin.

À mes cousins,

Pour la chaleur, la bonne humeur et le soutien que vous avez toujours su m'apporter.

À mon cher chat Sushi,

Pour la douceur et l'ambiance apaisante que tu as su apporter au quotidien.

Et enfin, à mon binôme Aymen,

Merci pour ton sérieux et ta collaboration, qui ont grandement enrichi cette aventure.

Dhia SOUDANI

Dédicaces

Je dédie ce travail à toutes les personnes qui m'ont accompagné et soutenu tout au long de ce parcours.

À mon père, dont la sagesse, les conseils avisés et le soutien constant ont toujours été une lumière dans les moments difficiles. Ta confiance en moi a été une source inestimable de motivation.

À ma mère, pour ton amour sans condition, ta patience et tes prières qui ont toujours été mon refuge et mon pilier. Sans ta présence rassurante, je n'aurais pas pu avancer avec autant de sérénité.

À mes frères, merci pour votre encouragement, vos mots réconfortants et votre bonne humeur, qui m'ont aidé à garder le moral même dans les moments compliqués.

À mon oncle, merci d'avoir toujours été là pour m'encourager et me motiver, même dans les moments difficiles.

À mon ami Youssef, pour ton écoute attentive, ta disponibilité et ta fidélité. Ton soutien a souvent renforcé ma confiance quand le doute s'installait.

Enfin, à mon binôme Dhia, pour ton sérieux, ta persévérance et ta collaboration sans faille. Travailler à tes côtés a été une expérience riche et précieuse.

Aymen MANSOURI

Remerciements

Nous souhaitons tout d'abord exprimer notre profonde gratitude à ***Dieu****, dont la guidance et le soutien nous ont accompagnés à chaque étape de ce projet. Sa grâce nous a permis de faire preuve de force, de patience et de persévérance pour mener à bien ce travail.*

Nous adressons nos remerciements les plus sincères à ***Monsieur Mohamed GLAOUI****, notre encadrant académique, pour son suivi rigoureux, ses conseils pertinents et la richesse de son encadrement. Ses interventions ont joué un rôle déterminant dans la bonne orientation de notre projet.*

Nous tenons également à exprimer notre profonde reconnaissance au ***Centre 4C de l'ISI Ariana****, et en particulier à* ***Monsieur Walid Miladi*** *ainsi qu'à* ***notre collègue Wassim Ben Abdallah****, pour leur disponibilité tout au long de la réalisation de ce projet.*

Enfin, nous remercions chaleureusement l'ensemble des ***enseignants de l'Institut Supérieur d'Informatique de l'Ariana (ISI Ariana)*** *pour la qualité de l'enseignement dispensé durant notre formation, ainsi que les* ***membres du jury*** *pour le temps et l'attention consacrés à l'évaluation de notre travail.*

Table des matières

Introduction générale **1**

1 Contexte général **3**

1.1 Organisme d'accueil 4

1.2 Étude et critique de l'éxistant 4

1.2.1 Définition d'un robot 4

1.2.2 Les robots assistants humanoïdes 5

1.2.3 Les fonctionnalités des robots assistants humanoïdes 5

1.2.4 Les robots assistants humanoïdes les plus connus 6

1.2.5 Analyse critique des solutions existantes dans la robotique humanoïde 7

1.3 Problématique 8

1.4 Solution proposée et objectifs globaux du projet 8

1.5 Méthodologie de travail et planification 9

1.5.1 Définition d'une méthodologie 9

1.5.2 Choix de la méthode Scrum 9

1.5.3 Répartition des rôles dans Scrum 9

1.5.4 Les étapes clés de Scrum 10

1.6 Conclusion 12

2 Analyse et spécification des besoins **13**

2.1 Identification des besoins 14

2.1.1 Besoins fonctionnels 14

2.1.2 Besoins non fonctionnels 14

2.2 Architecture 15

2.2.1 Tête du robot 15

2.2.2 Tronc 15

2.2.3 Base roulante du robot 15

2.3 Conception du systéme 17

2.3.1 Diagramme de cas d'utilisation 17

2.3.2 Diagramme de séquences . . . 18
2.3.3 Diagramme de classes . . . 19
2.4 Conception mécanique . . . 20
2.4.1 Vue éclatée . . . 20
2.4.2 Vue assemblée . . . 21
2.5 Conclusion . . . 21

3 Environnement logiciel et matériel 22
3.1 Environnement matériel . . . 23
3.1.1 Conception Matérielle du systéme . . . 23
3.1.2 Les périphérique . . . 23
3.2 Environnement logiciel . . . 25
3.2.1 Outils de dévelopement . . . 25
3.2.2 Librairies . . . 26

4 Réalisation, limites et perspectives 30
4.1 Méthodologie d'intégration des composants . . . 31
4.2 Simulation du circuit électronique . . . 32
4.3 câblage du circuit . . . 33
4.4 Développement logiciel embarqué du robot assistant . . . 33
4.4.1 Programmation à distance du Raspberry Pi avec VS Code (Remote-SSH) . . . 33
4.4.2 Installation de la base de données SQLite sur le Raspberry Pi . . . 34
4.4.3 Lecture des QR codes et affichage via la base de données SQLite . . . 36
4.4.4 Présentation de l'application embarquée . . . 38
4.4.5 Interface Web de contrôle et de visualisation . . . 44
4.4.6 L'emboîtement du robot assistant . . . 46
4.5 Limites du projet . . . 50
4.5.1 Fonctionnalités non implémentées . . . 50
4.5.2 Contraintes techniques et environnementales . . . 51
4.6 Perspectives d'amélioration . . . 51

Conclusion générale 53

Bibliographie **54**

Annexes **56**

Table des figures

2.1 Architecture globale 16
2.2 Diagramme de cas d'utilisation 18
2.3 Diagramme de séquence 19
2.4 Diagramme de classes 20
2.5 Vue éclatée du robot assistant humanoïde 21
2.6 Vue assemblée du robot assistant humanoïde 21

3.1 Composants du circuit électrique du robot assistant humanoïde 23

4.1 Simulation du câblage du circuit du robot assistant 32
4.2 Câblage réel du robot assistant humanoïde 33
4.3 Connexion SSH à distance entre VS Code et Raspberry Pi 34
4.4 Exécution d'un fichier Python sur le Raspberry Pi à distance à l'aide de VS Code (SSH) 34
4.5 Téléchargement et installation de SQLite3 via le terminal sur Raspberry Pi 35
4.6 Code SQL de création de la table `badges` sous SQLite 35
4.7 Structure de la table `badges` dans la base de données SQLite 36
4.8 Affichage en temps réel des informations d'un QR code détecté 38
4.9 Schéma de traitement d'une question vocale par le robot assistant 43
4.10 Interface du dashboard des capteurs sur la page web 45
4.11 Interface de contrôle du robot et affichage du dernier message de l'assistant 45
4.12 Exemple de fichiers d'historique des conversations générés automatiquement 46
4.13 Modélisation 3D de l'emboîtement du robot assistant (vue éclatée) sous Fusion 360 . 47
4.14 Modélisation 3D de l'emboîtement du robot assistant (vue assemblée) sous Fusion 360 47
4.15 Imprimante 3D utilisée au sein du centre 4C 48
4.16 Phase d'impression 3D de l'emboîtement du robot 48
4.17 Composants imprimés en 3D après la fabrication 49
4.18 Robot humanoïde final assemblé avec les composants imprimés en 3D 49
4.19 Vue latérale gauche 50
4.20 Vue latérale droite 50

Liste des tableaux

1.1 Répartition des rôles dans l'équipe Scrum 10
1.2 Backlog Produit – Robot 11

3.1 Composants principaux du robot assistant humanoïde avec description détaillée et fonctionnement 24

Liste des abréviations

— **API** : Application Programming Interface (Interface de Programmation Applicative)

— **DHT11** : Capteur de température et d'humidité numérique

— **EEA** : Électronique, Électrotechnique et Automatisme

— **Flask** : Micro-framework web Python utilisé pour créer l'application serveur

— **GPIO** : General Purpose Input/Output (Entrée/Sortie à Usage Général)

— **HC-SR04** : Capteur ultrasonique de distance

— **IA** : Intelligence Artificielle

— **JSON** : JavaScript Object Notation (Format d'échange de données)

— **LCD** : Liquid Crystal Display (Écran à cristaux liquides)

— **MQ2** : Capteur de gaz combustible (monoxyde de carbone, gaz inflammables)

— **MJPEG** : Motion JPEG (Format vidéo compressé)

— **PFE** : Projet de Fin d'Études

— **QR Code** : Quick Response Code (Code réponse rapide)

— **RAS** : Rien À Signaler

— **Raspberry Pi (RPI)** : Micro-ordinateur utilisé dans le projet

— **SDK** : Software Development Kit (Kit de développement logiciel)

— **SocketIO** : Bibliothèque facilitant la communication temps réel client-serveur

— **USB** : Universal Serial Bus (Bus universel en série)

— **gTTS** : Google Text-to-Speech (Synthèse vocale de Google)

— **Ctrl+C** : Commande clavier pour interruption manuelle d'un programme

Introduction générale

Dans un monde où la technologie évolue rapidement, l'intelligence artificielle et la robotique prennent de plus en plus de place dans notre vie quotidienne. Les robots, autrefois limités aux usines, se retrouvent aujourd'hui dans les maisons, les écoles, les hôpitaux ou encore les commerces. Ils sont conçus pour aider les humains dans leurs tâches habituelles, en particulier celles qui sont répétitives, fatigantes ou peu valorisantes.

Aujourd'hui, d'après une étude de l'*IFR* (International Federation of Robotics), le nombre de robots de service – qu'ils soient personnels ou professionnels – connaît une croissance de plus de 30 % par an à l'échelle mondiale. Cette évolution rapide montre à quel point la robotique intelligente prend une place importante dans notre quotidien, que ce soit dans le domaine de la santé, de l'éducation, de la sécurité ou de l'assistance aux personnes.

Malheuresement en Tunisie, cette avancée reste encore très limitée. Nos méthodes de travail restent souvent traditionnelles, et la robotique y est encore peu développée. Elle se limite principalement à des projets académiques ou à quelques expériences isolées. Le manque d'investissement, l'absence d'infrastructures spécialisées et la faible intégration de l'intelligence artificielle dans les solutions locales freinent son développement. Pourtant, les besoins sont bien là, et le potentiel est énorme.

Dans ce contexte, nous réalisons notre stage de fin d'études au sein de l'Institut Supérieur de l'Informatique dans le but d'obtenir le diplôme national en licence en Électronique, Électrotechnique et Automatique . Ce projet consiste à concevoir et réaliser un robot assistant humanoïde capable d'aider dans certaines tâches quotidiennes, tout en proposant des fonctionnalités utiles au sein même de notre établissement, afin de faciliter la vie des étudiants.

Dans le premier chapitre, nous présentons l'organisme d'accueil, la problématique, la solution proposée et la méthodologie de travail. Le deuxième chapitre est dédié à la conception, avec une description de l'architecture et des besoins à l'aide de diagrammes UML. Le troisième chapitre décrit l'environnement matériel et logiciel utilisé. Enfin, Dans le quatrième chapitre, nous détaillons les étapes de réalisation du robot assistant, depuis le câblage jusqu'à la programmation, en abordant

également les limites rencontrées durant le développement ainsi que les perspectives d'amélioration et d'évolution futures du système.

Chapitre 1

Contexte général

Plan

1 **Organisme d'accueil** 4

2 **Étude et critique de l'éxistant** 4

3 **Problématique** 8

4 **Solution proposée et objectifs globaux du projet** 8

5 **Méthodologie de travail et planification** 9

6 **Conclusion** 12

Introduction

Ce chapitre débute par la présentation de l'organisme d'accueil du projet. Ensuite, nous définissons la robotique humanoïde et présentons quelques-uns des robots assistants les plus connus. Une analyse critique de l'existant permettra d'identifier les limites des solutions actuelles et de mieux situer notre projet dans ce contexte. Nous poursuivons avec la formulation de la problématique, la solution proposée pour y répondre, ainsi que la méthodologie de travail adoptée.

1.1 Organisme d'accueil

L'Institut Supérieur d'Informatique (ISI) est un établissement public d'enseignement supérieur situé à l'Ariana, en Tunisie. Il est intégré à l'Université de Tunis El Manar et a été créé officiellement par le décret du 14 août 2001. L'Institut a pour mission principale la formation d'un personnel qualifié dans les domaines de l'informatique, des systèmes embarqués, du génie logiciel et des nouvelles technologies. Il délivre une formation académique de haute qualité alliant enseignement théorique et travaux pratiques en réalisant des projets, des stages et des travaux de recherche. L'institut offre aussi différents services pour accompagner la communauté étudiante : encadrement des projets de fin d'études, accès à des laboratoires entièrement équipés et participation à des activités scientifiques, culturelles et technologiques. Grâce à son ambiance dynamique et à ses relations avec les acteurs du secteur technologique, l'ISI constitue un cadre propice à l'innovation et au développement des compétences professionnelles.

1.2 Étude et critique de l'éxistant

Cette section propose une définition des robots et présente les robots assistants humanoïdes les plus connus, avant d'en analyser les points forts et les limites afin de mieux orienter notre propre conception.

1.2.1 Définition d'un robot

Un robot est un système mécatronique -combinant mécanique, électronique et informatique- conçu pour exécuter automatiquement des tâches qui reproduisent ou imitent des actions humaines dans un domaine spécifique. La conception de ces systèmes relève d'une discipline scientifique appelée robotique, une branche de l'automatique [1] .

1.2.2 Les robots assistants humanoïdes

Face à la technologie, il est difficile de ne pas être curieux quant au fonctionnement d'un appareil — et cela est d'autant plus vrai lorsqu'il s'agit d'un robot humanoïde, véritable concentré de technologies. Pour rappel, un robot humanoïde est un robot conçu pour ressembler à un être humain, avec généralement une tête, un torse, deux bras et deux jambes. Cependant, il arrive souvent que l'appareil ne reproduise qu'une seule partie de l'anatomie humaine. À première vue, concevoir un tel engin peut sembler complexe — et cela l'est véritablement — car il incarne la synthèse de nombreux savoirs humains. Les robots humanoïdes ne sont pas fondamentalement différents des autres robots : comme eux, ils nécessitent des capteurs, des actionneurs et un système embarqué pour fonctionner. L'ensemble est monté sur une structure qui supporte le poids de la machine et protège ses composants internes [2].

1.2.3 Les fonctionnalités des robots assistants humanoïdes

Ces robots intègrent plusieurs fonctions clés visant à assister les utilisateurs dans leur quotidien de manière intelligente et interactive.

1.2.3.1 Assistance à la personne

Les robots humanoïdes peuvent assister les personnes âgées et handicapées en fournissant des rappels, en surveillant leur bien-être et en facilitant certaines actions du quotidien [3].

1.2.3.2 Éducation et tutorat

Ces robots sont utilisés pour enseigner de manière interactive dans divers domaines (langues, mathématiques, sciences) et pour aider les élèves à réviser ou comprendre des matières en fournissant des réponses immédiates et des interactions dynamiques [3].

1.2.3.3 Industrie et service client

Dans des lieux comme les magasins, musées ou aéroports, les robots humanoïdes servent d'hôtes, fournissent des informations et aident à la gestion des services. Ils sont également utilisés lors d'événements ou expositions pour interagir avec les participants et réaliser des démonstrations[3].

1.2.3.4 Recherche et développement

Les robots humanoïdes sont utilisés dans les laboratoires pour tester de nouveaux produits ou simuler des interactions humaines. Ils servent aussi dans des environnements extrêmes (catastrophes naturelles, missions spatiales) pour effectuer des tests ou collecter des données [3].

1.2.3.5 Sécurité

Ils peuvent patrouiller dans des zones publiques ou privées pour assurer la sécurité, détecter des comportements suspects et signaler des incidents [3].

1.2.3.6 Soins médicaux et rééducation

Les robots humanoïdes aident dans la rééducation physique des patients en les guidant dans leurs exercices et assistent le personnel hospitalier en livrant des médicaments et en fournissant des informations aux patients [3].

1.2.4 Les robots assistants humanoïdes les plus connus

Parmi les robots humanoïdes les plus connus, Pepper, développé par SoftBank Robotics, est le premier robot social capable de reconnaître les visages et les émotions humaines. Optimisé pour l'interaction, il est utilisé dans plus de 2000 entreprises à travers le monde pour accueillir, informer et guider les visiteurs [4]. Du même constructeur, Nao est un robot autonome et programmable, utilisé principalement à des fins éducatives et de recherche dans plus de 50 pays [5]. ASIMO, conçu par Honda, est reconnu comme le premier robot humanoïde capable de marcher de manière réaliste. Il a été utilisé dans des démonstrations publiques et loué par des entreprises comme IBM, avec pour ambition future d'assister les personnes handicapées ou âgées [6]. De son côté, Atlas, développé par Boston Dynamics avec le soutien de la DARPA, est destiné aux missions de recherche et de sauvetage et se distingue par sa robustesse et sa mobilité impressionnante [7]. Enfin, Sophia, conçue par Hanson Robotics et activée en 2015, est une gynoïde capable d'interagir de façon intelligente avec les humains ; elle est devenue célèbre après avoir obtenu la nationalité saoudienne, une première mondiale [8].

1.2.5 Analyse critique des solutions existantes dans la robotique humanoïde

Écosystème fermé : L'un des problèmes qu'on retrouve souvent avec les robots assistants humanoïdes actuels, c'est qu'ils sont assez fermés. En gros, on est un peu « bloqué » dans leur propre système : il faut utiliser leurs outils, suivre leurs règles, et ce n'est pas toujours simple d'adapter le robot à ce qu'on veut vraiment faire. Si on a des besoins spécifiques ou qu'on veut ajouter des capteurs ou des fonctions, ça devient vite compliqué. En plus, même si ces robots sont censés être simples à utiliser, leur mise en place ou leur programmation demande quand même pas mal de connaissances. Il faut parfois apprendre un langage bien précis ou utiliser des logiciels pas forcément accessibles à tout le monde. Et quand on n'a pas un bon niveau en informatique, ça peut être décourageant. C'est dommage, parce que ces robots ont beaucoup de potentiel, mais toutes ces barrières freinent leur adoption par le grand public ou certaines institutions.

Complexité d'utilisation : Même si ces robots sont très avancés sur le plan technologique, ils restent souvent compliqués à utiliser, surtout pour les personnes qui ne sont pas à l'aise avec l'informatique. Les outils pour les programmer ou les contrôler ne sont pas toujours faciles à comprendre, et rien que la première configuration peut demander des compétences techniques assez poussées. Par exemple, des robots comme *Nao* ou *Pepper* nécessitent souvent de connaître un langage de programmation ou d'utiliser des plateformes spécifiques. Tout ça rend leur prise en main difficile pour le grand public. Résultat : seuls les experts ou les chercheurs arrivent vraiment à en tirer parti. En plus, certains de leurs services ne fonctionnent que si on a un bon ordinateur et une connexion internet stable, ce qui complique encore plus leur utilisation dans certains endroits ou contextes.

Coût élevé : Des modèles comme *Pepper* ou *Nao* nécessitent un investissement conséquent, ce qui les rend difficilement accessibles pour les écoles et les petites structures. Et ce n'est pas seulement le prix d'achat qui est considéré comme un problème : la maintenance, les mises à jour logicielles, ou encore l'ajout des nouvelles fonctionnalités représentent aussi des frais non négligeables. À cause de cela, ces solutions restent souvent réservées à des organismes bien financés, ce qui limite leur utilisation à grande échelle.

Inadaptation au contexte local : Notre faculté ne dispose ni d'un système autonome de détection d'incendie, ni d'un système automatisé de gestion des entrées lors des événements. Les tâches d'accueil et de filtrage sont généralement assurées par des étudiants, ce qui mobilise inutilement

du personnel et entraîne parfois des erreurs ou des retards. De plus, les nouveaux étudiants ou visiteurs n'ont souvent pas accès à des informations claires et interactives sur l'établissement ou ses filières. Les robots existants n'intègrent pas ce type de fonctions orientées vers les besoins spécifiques d'un établissement universitaire tunisien comme le nôtre.

1.3 Problématique

Bien que la robotique sociale ait connu des avancées notables ces dernières années, son intégration dans les milieux éducatifs et institutionnels reste marginale. Les robots humanoïdes actuellement disponibles se heurtent à plusieurs obstacles : leur manque d'ouverture limite toute personnalisation, leur prix reste inaccessible pour la majorité des établissements, et leur prise en main requiert souvent des compétences techniques élevées. Dans un contexte comme celui de notre faculté, ces limitations deviennent particulièrement visibles. Il n'existe pas de système automatisé de filtrage des visiteurs lors des événements ; l'identification des participants repose encore sur des méthodes manuelles, mobilisant des étudiants et engendrant des erreurs. De plus, l'accueil des nouveaux arrivants est peu structuré, et les informations sur les filières ou les services proposés restent difficiles d'accès sans assistance humaine. Par ailleurs, bien qu'un système de détection d'incendie soit en place, il n'est pas intelligent ni interconnecté avec d'autres dispositifs pouvant déclencher des alertes ou orienter les personnes en cas d'urgence. Face à ces constats, il est impératif de concevoir une solution robotique accessible, évolutive et adaptée au contexte local, capable de combler ces lacunes à travers des fonctionnalités telles que l'assistance interactive, la gestion automatisée des entrées par QR code, et la détection intelligente d'incendie.

Face à ces constats, il devient nécessaire de proposer une solution plus accessible, modulable et adaptée aux besoins réels du terrain éducatif.

1.4 Solution proposée et objectifs globaux du projet

Pour répondre aux défis identifiés, notre projet de robot assistant humanoïde propose une solution accessible, flexible et adaptée aux besoins spécifiques de notre faculté. Cette approche vise à surmonter les limitations des systèmes existants, en offrant une alternative plus économique et plus simple à utiliser. Le robot sera conçu pour faciliter l'accueil des visiteurs et la gestion des événements, grâce à des fonctionnalités comme la détection automatique des participants par QR code, tout en étant capable de fournir des informations sur les filières et les services de l'établissement. En outre,

bien que la faculté dispose d'un système de détection d'incendie, notre robot ajoutera une dimension intelligente à cette fonction en offrant une surveillance continue et des alertes en cas d'urgence. Ainsi, l'objectif global de ce projet est de créer un robot assistant humanoïde qui non seulement répond à des besoins pratiques, mais qui soit également facilement accessible et utilisable, même pour des utilisateurs sans compétences techniques avancées. Le robot sera conçu pour être évolutif, permettant des ajouts de fonctionnalités en fonction des besoins futurs, et pour offrir une solution plus abordable par rapport aux modèles existants.

1.5 Méthodologie de travail et planification

Cette section décrit l'approche méthodologique adoptée pour la réalisation du projet, ainsi que la planification des différentes étapes de développement afin d'assurer une progression structurée et efficace.

1.5.1 Définition d'une méthodologie

Afin de réaliser un systéme de haute qualité, il est important de choisir un ensemble structuré de méthodes, principes et régles pour répondre aux besoins en termes de délais et de couts.Cette section nous permet à indiquer comment planifier , éxecuter et controler les tàches.

1.5.2 Choix de la méthode Scrum

Scrum est une méthode agile de gestion de projet, axée sur l'amélioration continue, la réactivité face aux changements et la collaboration. Elle favorise des cycles courts et itératifs, intégrant régulièrement les retours du client pour optimiser le produit. Elle permet aussi à chaque acteur du projet d'avoir une vue claire sur son avancement, tout en réduisant les anomalies et en ajustant les priorités au fil du temps [9].

1.5.3 Répartition des rôles dans Scrum

Cette partie présente les différents rôles définis par la méthode Scrum et leur répartition au sein de notre équipe afin d'assurer une organisation claire et collaborative du projet [9].

1.5.3.1 Maître du Scrum

Le Maître du Scrum veille au respect de la méthode Scrum. Il facilite la communication, résout les obstacles et aide l'équipe à gagner en efficacité, sans jouer un rôle hiérarchique [9].

1.5.3.2 Responsable du Produit

Le Responsable Produit représente le client. Il définit les fonctionnalités, les priorise, valide les livrables et s'assure que le produit final correspond aux besoins. Il est l'intermédiaire principal entre le client et l'équipe [9].

1.5.3.3 Équipe de développement

l'équipe est auto-organisée et sans hiérarchie formelle. Chaque membre contribue au sprint selon ses compétences, dans un esprit de collaboration. .

Tableau 1.1 : Répartition des rôles dans l'équipe Scrum

Rôle	**Nom(s)**
Maître du Scrum	MOHAMED GLAOUI
Responsable du Produit	Université (Équipe de production)
Équipe de développement	DHIA SOUDANI, AYMEN MANSOURI

1.5.4 Les étapes clés de Scrum

Cette section détaille les principales étapes du cadre Scrum que nous avons suivies tout au long du projet.

1.5.4.1 Backlog produit

Le Responsable du produit collecte les besoins du client sous forme de « user stories » dans un backlog. Cette liste est évolutive et guide les priorités du projet.

Tableau 1.2 : Backlog Produit – Robot

ID	Cas d'utilisation	Histoire utilisateur	Priorité
1	Afficher la météo (via l'API Météo)	En tant qu'utilisateur, je souhaite consulter la météo via une API pour connaître les conditions actuelles.	Élevée
2	Scanner un code QR pour la reconnaissance du badge	En tant qu'utilisateur, je souhaite scanner un code QR pour que le robot puisse reconnaître mon badge.	Très élevée
3	Interaction vocale (ChatGPT)	En tant qu'utilisateur, je souhaite interagir vocalement avec le robot pour recevoir des réponses intelligentes (via ChatGPT).	Très élevée
4	Contrôler les moteurs CC pour le déplacement	En tant qu'utilisateur, je souhaite contrôler les moteurs CC du robot pour qu'il puisse se déplacer.	Très élevée
5	Afficher le flux vidéo (caméra Raspberry Pi)	En tant qu'utilisateur, je souhaite visualiser le flux vidéo de la caméra Raspberry Pi pour surveiller l'environnement du robot.	Élevée
6	Reconnaissance vocale	En tant qu'utilisateur, je souhaite que le robot comprenne mes instructions vocales pour interagir sans interface physique.	Élevée

1.5.4.2 Sprint

Le projet est découpé en sprints de deux semaines. À chaque sprint, l'équipe sélectionne des tâches du backlog et s'engage à livrer une ou plusieurs fonctionnalités prêtes à l'emploi [9].

1.5.4.3 Mêlée quotidienne

Chaque jour, l'équipe se réunit brièvement (5–10 minutes) pour faire le point sur l'avancement, identifier les blocages et planifier la journée [9].

1.5.4.4 Revue de Sprint

En fin de sprint, l'équipe présente les fonctionnalités développées au Product Owner, qui les valide ou propose des ajustements selon les besoins du client [9].

1.5.4.5 Rétrospective

L'équipe analyse les points forts, les difficultés et les axes d'amélioration. Cette étape favorise l'adaptation et l'efficacité au fil des sprints [9].

1.5.4.6 Scrum : une méthode au service de l'efficacité

Scrum est une méthode simple en apparence,mais qui nécessite rigueur et discipline. Son succès est basé sur l'engagement de l'équipe, l'adaptabilité, et l'amélioration continue. Elle permet d'obtenir un produit évolutif, en adéquation avec les attentes réelles du client.

1.6 Conclusion

Au cours de ce premier chapitre , nous avons présenté le contexte général des robots assistants humanoïdes. cette étude des solutions existantes, nous a permis d'identifier plusieurs limites importantes. Ces constats nous ont aidés à définir une problématique claire , à laquelle notre projet vise d'apporter une solution adéquate en proposant un robot moins complexe, plus raisonnable et plus adapté aux besoins locaux, de plus nous avons expliqué le choix méthodologique. Dans le chapitre suivant , nous nous concentrerons sur la spécification des besoins et la conception.

Chapitre 2

Analyse et spécification des besoins

Plan

1 Identification des besoins 14

2 Architecture 15

3 Conception du systéme 17

4 Conception mécanique 20

5 Conclusion 21

Introduction

Dans ce deuxième chapitre, nous détaillons les besoins fonctionnels et non fonctionnels, présentons une vue globale sur le fonctionnement du robot à travers les diagrammes de cas d'utilisation, de séquences et de classes, puis décrivons l'architecture générale ainsi que la conception mécanique de notre robot assistant humanoïde.

2.1 Identification des besoins

cette partie a comme but d'identifier un cadre précis qui guide les choix fonctionnels et techniques afin que le robot assistant réponde aux besoins et aux attentes des utilisateurs finaux.

2.1.1 Besoins fonctionnels

les besoins fonctionnels présentent les services que le systéme doit offrir aux utilisateurs ,ce sont les princiales tàches comme l'identification des utilisateurs, l'affichage des données environnementales, ou encore la communication vocale.

Notre projet de robot assistant humanoïde offre des fonctionnalités simples mais efficaces :

- **Reconnaissance des individus :** le robot pourra identifier les utilisateurs (étudiants ou enseignants) grâce à la lecture d'un code QR.
- **Affichage des données environnementales :** il affichera des informations comme la température, l'humidité et la présence éventuelle de gaz.
- **Informations météorologiques :** les données météo seront fournies en temps réel à l'aide d'une API connectée.
- **Communication vocale :** il pourra interagir vocalement avec l'utilisateur de façon naturelle et fluide.
- **Déplacement contrôlé :** le robot pourra être dirigé via une interface web ou à l'aide de commandes vocales.

2.1.2 Besoins non fonctionnels

Dans cette partie , nous allons aborder comment le systéme doit fonctionner , les besoins non fonctionnels contiennent les exigences liées aux performances et l'utilisabilité du systéme pour garantir que ce dernier fonctionne de maniére optimale.

- **Facilité d'utilisation :** Le robot doit être simple à utiliser, même pour des personnes sans compétences techniques.
- **Interface web claire :** L'interface web doit être claire et simple à comprendre.
- **Commandes vocales accessibles :** Les commandes vocales doivent être simple et claires.
- **Réactivité du système :** Le système doit fournir des réponses rapides surtout quand il s'agit de parler avec l'utilisateur ou d'afficher des informations.
- **Utilisation de logiciels open source :** Le robot doit être construit avec des logiciels open source ce qui rend les modifictaions ou les améliorations futures plus faciles.
- **Maintenance facilitée :** Les composants doivent être facilement démontables et remplaçables pour simplifier la maintenance et l'évolution du robot si besoin.

2.2 Architecture

L'architecture présente d'une façon globale notre robot assistant humanoide afin de montrer l'organisation des différentes composants logiciels et matériels dans le but de satisfaire les besoins identifiés précédement. le robot asssistant humanoïde est divisé en 3 parties :

2.2.1 Tête du robot

- **Raspberry Pi 4 :** unité de traitement centrale.
- **Caméra Raspberry Pi Module 2 :** détecte le QR code des utilisateurs.
- **Capteur DHT11 :** mesure la température, l'humidité et détecte la présence éventuelle de gaz.
- **Capteur gaz MQ-5 :** détecte des gaz inflammables.
- **Écran LCD I2C 16x2 :** affichage local des données.
- **Micro et Haut-parleur USB :** pour l'interaction vocale avec l'utilisateur.

2.2.2 Tronc

- Support structurel pour le câblage.

2.2.3 Base roulante du robot

- **Deux roues motrices :** assurent le déplacement du robot.

- **Une roue libre :** placée à l'arrière pour maintenir la stabilité du robot.
- **2 Moteurs DC 7V :** convertissent l'énergie électrique en énergie mécanique pour entraîner les roues.
- **Module L298N :** contrôle la vitesse et le sens de rotation des moteurs à courant continu.
- **Capteur à ultrasons HC-SR04 :** détection d'obstacles.
- **Batterie :** alimentation électrique du robot.

La **figure 2.1** illustre l'architecture globale du robot assistant humanoïde, en mettant en évidence l'interaction entre les différentes composantes matérielles et logicielles, ainsi que leur lien avec l'utilisateur. Ce schéma permet de mieux comprendre la structure du système et la manière dont ses éléments interagissent pour assurer le bon fonctionnement du robot.

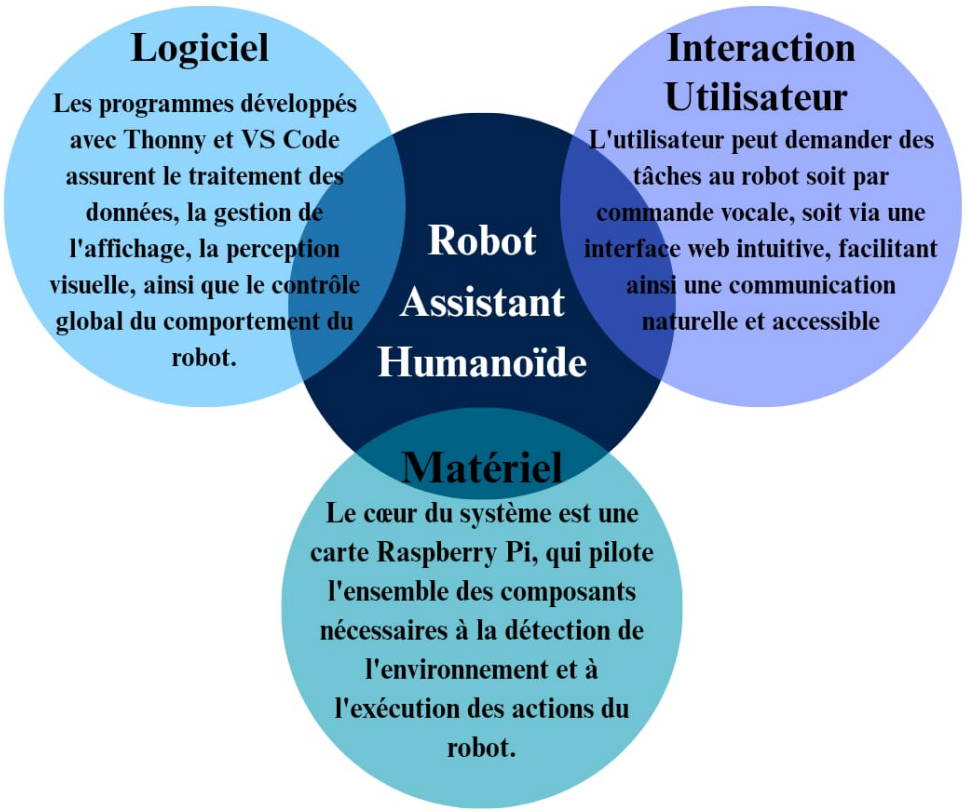


Figure 2.1 : Architecture globale

2.3 Conception du systéme

Le langage UML (Unified Modeling Language) est un langage visuel standard, riche sur les plans sémantique et syntaxique, principalement utilisé pour modéliser l'architecture, la conception et la mise en œuvre de systèmes logiciels complexes. Il permet de représenter à la fois la structure et le comportement d'un système. Bien que largement utilisé dans le développement logiciel, UML s'applique également à d'autres domaines, comme la modélisation des processus industriels. À l'instar des plans d'architecture dans la construction, UML utilise différents types de diagrammes pour définir les frontières, la structure interne et le comportement dynamique d'un système. Bien qu'il ne s'agisse pas d'un langage de programmation, UML est souvent associé à des outils générant du code source dans divers langages et est étroitement lié à l'approche orientée objet [10].

2.3.1 Diagramme de cas d'utilisation

En langage UML, les diagrammes de cas d'utilisation modélisent le comportement d'un système et permettent de collecter les exigences du système. Les diagrammes de cas d'utilisation décrivent les fonctions générales et la portée d'un système.Ces diagrammes permettent d'identifier les interactions entre le système et ses acteurs. Ces diagrammes montrent ce que le système accomplit et comment les acteurs interagissent avec lui, sans détailler son fonctionnement interne[11].

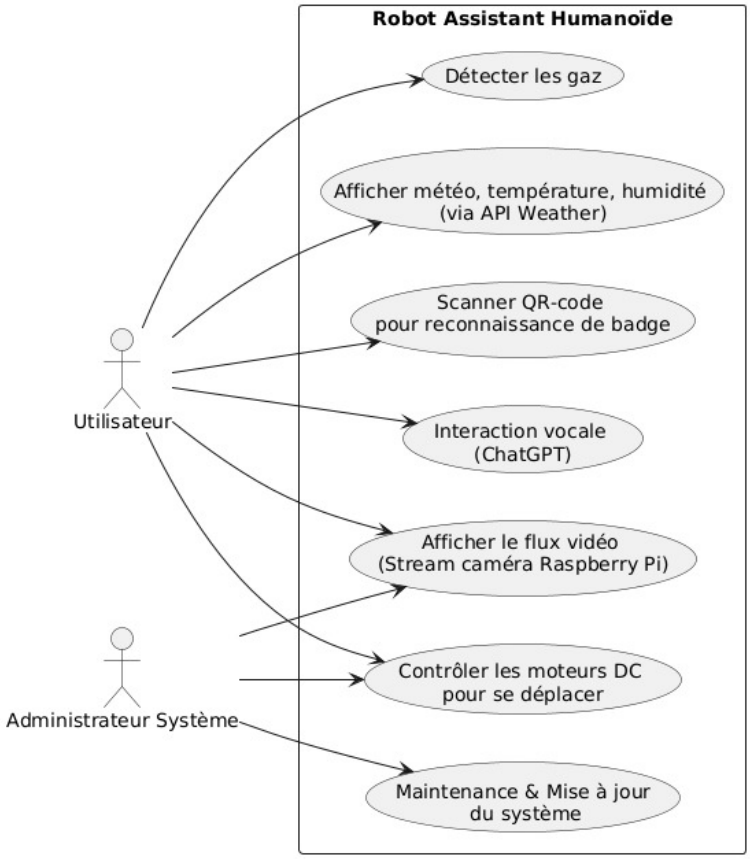


Figure 2.2 : Diagramme de cas d'utilisation

2.3.2 Diagramme de séquences

Les diagrammes de séquence sont une méthode couramment utilisée pour la modélisation dynamique en UML. Ils mettent l'accent sur les lignes de vie, les processus et les objets qui coexistent simultanément, ainsi que sur les échanges de messages entre eux, permettant ainsi de représenter l'exécution d'une fonction avant la fin de leur cycle de vie [12].

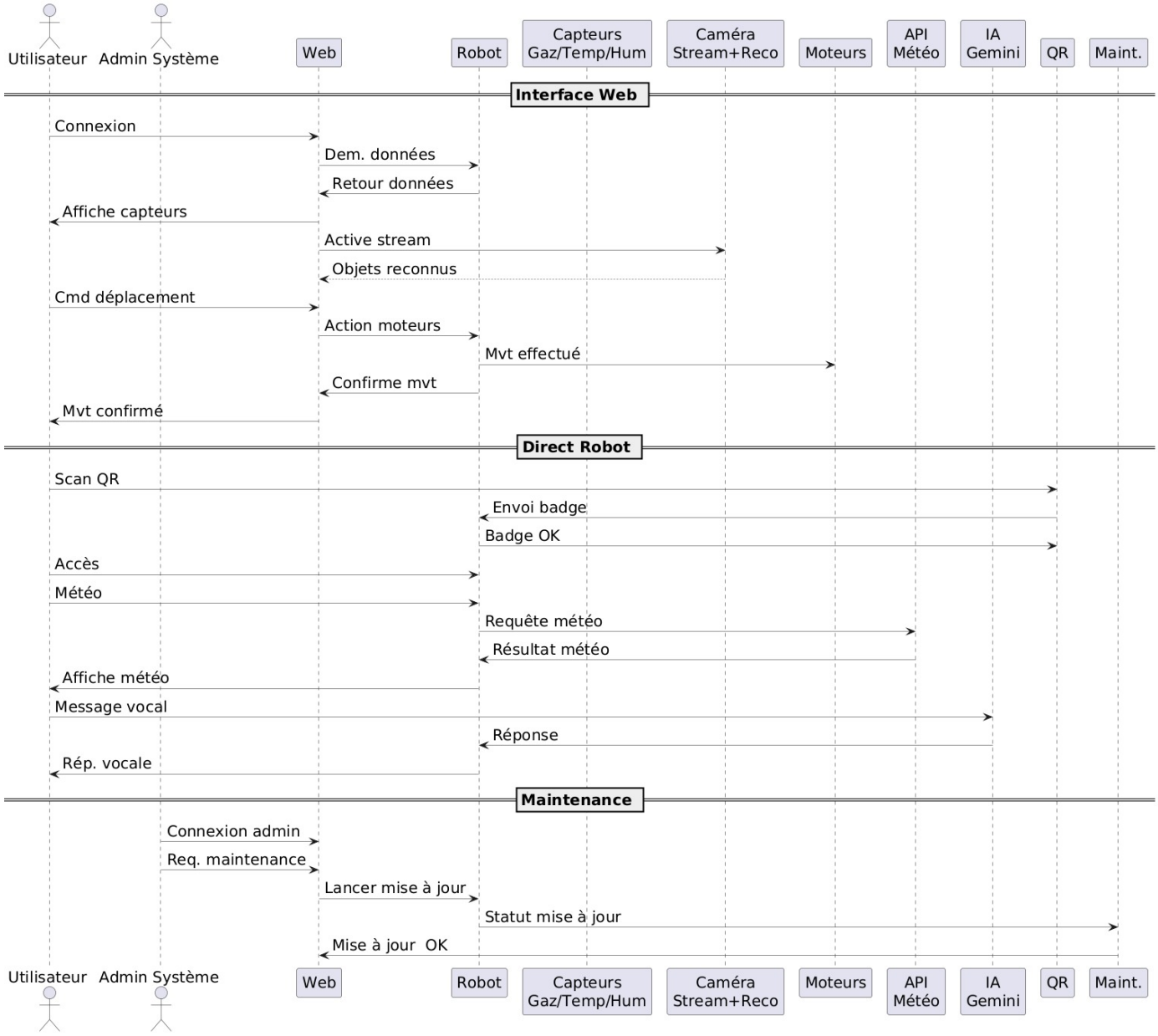


Figure 2.3 : Diagramme de séquence

2.3.3 Diagramme de classes

Les diagrammes de classes comptent parmi les types de diagrammes UML les plus essentiels, car ils représentent de manière claire la structure d'un système en modélisant ses classes, leurs attributs, leurs opérations, ainsi que les relations entre les objets correspondants [13].

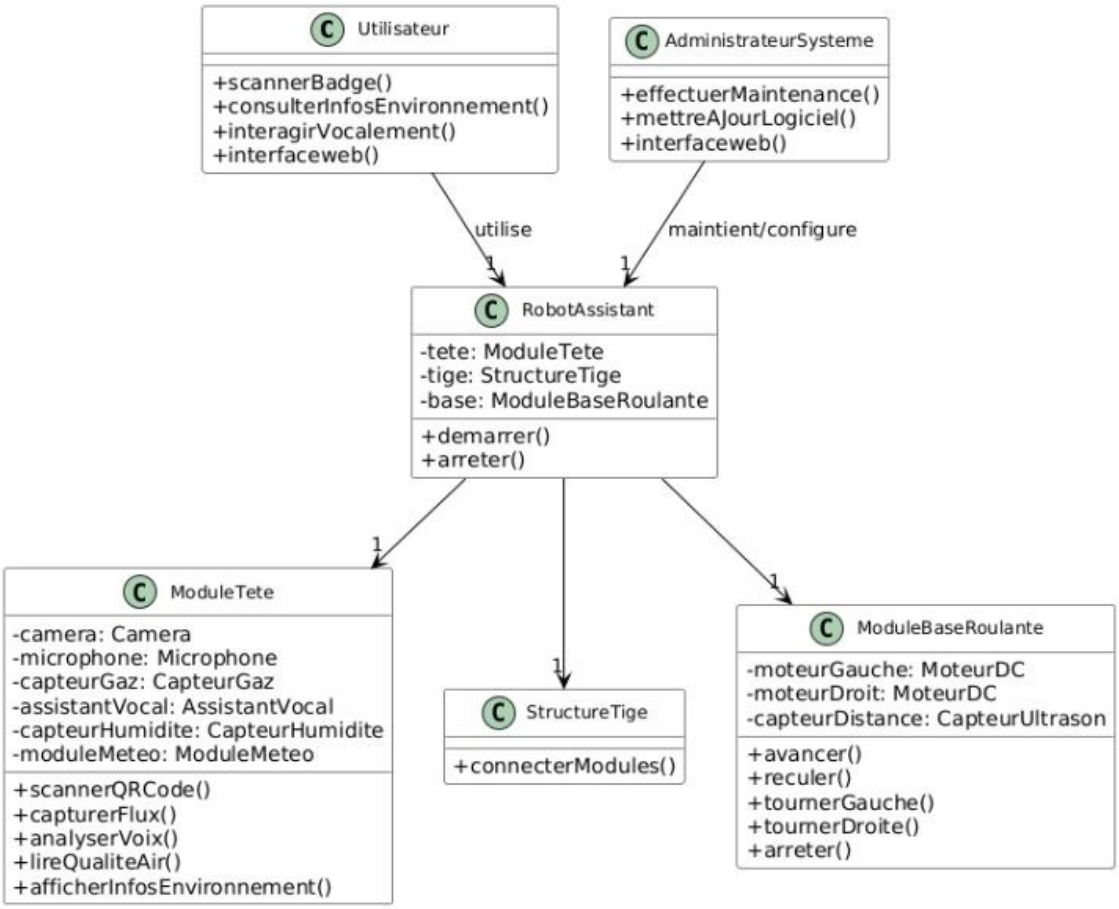


Figure 2.4 : Diagramme de classes

2.4 Conception mécanique

Nous présentons ici une vue éclatée ainsi qu'une vue assemblée du robot afin d'illustrer sa structure mécanique et l'agencement de ses composants.

2.4.1 Vue éclatée

La figure suivante montre la vue éclatée du robot, dans laquelle les différents composants mécaniques sont représentés de maniére séparée en se basant à leur position réelle dans l'assemblage.À l'aide de cette vue on peut mieux comprendre la structure interne du robot . On note la présence d'une Raspberry Pi 4, une caméra Raspberry Pi Module 2, un capteur DHT11, un capteur gaz MQ-5, un écran LCD I2C 16x2, ainsi qu'un micro et un haut-parleur USB. Le tronc sert de support structurel pour le câblage. La base roulante comprend deux roues motrices, une roue libre, deux moteurs DC 7V, un module L298N et un capteur à ultrasons HC-SR04.



Figure 2.5 : Vue éclatée du robot assistant humanoïde

2.4.2 Vue assemblée

La figure suivante montre la vue assemblée du robot assistant humanoïde, présentant l'aspect final du système quand toutes les pièces mécaniques montées. Cette vue nous aide à visualiser l'architecture générale du robot qui est composée de la Tête,Tronc et base roulante du robot .



Figure 2.6 : Vue assemblée du robot assistant humanoïde

2.5 Conclusion

Dans ce chapitre, nous avons abordé les besoins fonctionnels et non fonctionnels.Ainsi nous avons posé une architecture générale du robot et une conception du systeme à l'aide des differents diagrammes avant de terminer avec la conception mécanique. Dans le chapitre suivant nous allons étudier l'environnement matériel et logiciel du systéme.

Chapitre 3

Environnement logiciel et matériel

Plan

1 Environnement matériel 23

2 Environnement logiciel 25

Introduction

Dans ce chapitre, nous allons étudier et analyser l'environnement logiciel et matériel de ce projet avec les outils qui nous ont permis de le réaliser.

3.1 Environnement matériel

Dans cette section, nous détaillons l'environnement matériel utilisé pour réaliser le projet.

3.1.1 Conception Matérielle du systéme

cette figure présente les différents composants du circuit électrique à concevoir.

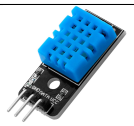
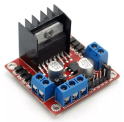


Figure 3.1 : Composants du circuit électrique du robot assistant humanoïde

3.1.2 Les périphérique

Ce tableau présente le matériel électronique principal utilisé pour construire le circuit électrique de notre robot assistant. Pour chaque composant, une description de son fonctionnement et de son rôle dans le système est indiquée, afin de mieux comprendre leur contribution au bon déroulement des opérations du robot.

Tableau 3.1 : Composants principaux du robot assistant humanoïde avec description détaillée et fonctionnement

Composant	Description avec fonctionnement	Image
Raspberry Pi 4	Cœur du système. Exécute le code principal et contrôle tous les périphériques connectés, assurant la coordination globale du robot.	
Camera Module 2	Capture d'images et vidéos. Utilisée pour la détection des visages et la lecture des QR codes afin d'identifier les utilisateurs ou objets.	
DHT11	Mesure la température et l'humidité ambiantes grâce à ses capteurs intégrés, transmettant les données au système pour affichage et analyse.	
Capteur de gaz	Détecte la présence de gaz nocifs dans l'environnement. Alerte le système en cas de détection pour prendre des mesures de sécurité.	
HC-SR04	Utilise des ultrasons pour mesurer la distance aux obstacles devant le robot, permettant d'éviter les collisions lors des déplacements.	
Écran LCD I2C	Affiche en temps réel les données mesurées par les capteurs (température, humidité, alertes, etc.) pour l'utilisateur.	
Moteur DC	Assure le mouvement des roues du robot, permettant la locomotion et le déplacement selon les commandes reçues.	
Module L298N	Contrôle la vitesse et le sens de rotation des moteurs DC en utilisant des signaux de commande PWM pour gérer les déplacements du robot.	

3.2 Environnement logiciel

Cette section décrit l'ensemble des outils, technologies et bibliothèques logicielles utilisés pour le développement du robot assistant humanoïde.

3.2.1 Outils de dévelopement

Nous présentons ici les principaux environnements et logiciels qui ont été utilisés pour programmer, tester et intégrer les différentes fonctionnalités du système.

3.2.1.1 Visual Studio Code (VS Code)

Visual Studio Code (VS Code) est un environnement de développement intégré (IDE) léger et modulable, développé par Microsoft. Il se distingue par sa polyvalence et sa capacité à s'adapter aux différents langages de programmation et systèmes d'exploitation, répondant ainsi aux besoins variés des développeurs[14].



Logo de Visual Studio Code (VS Code)

3.2.1.2 VNC VIEWER

VNC Viewer est un logiciel permettant de contrôler à distance un ordinateur exécutant un serveur VNC (VNC Server). Il permet à des appareils comme des ordinateurs, tablettes ou smartphones d'accéder à un autre ordinateur situé à distance, et d'en prendre le contrôle. Une fois connecté, VNC Viewer transmet les événements du clavier, de la souris ou du tactile au serveur VNC, permettant à l'utilisateur de contrôler l'ordinateur distant comme s'il était physiquement présent devant lui. Ce système est particulièrement utile pour accéder à un ordinateur à distance et travailler dessus de manière transparente, que ce soit à partir d'un téléphone mobile ou d'un autre ordinateur [15].



Logo de VNC Viewer

3.2.1.3 THONNY

Thonny est un environnement de développement intégré (IDE) gratuit et open-source, conçu principalement pour le langage de programmation Python,Thonny simplifie le processus de développement en fournissant tous les outils, bibliothèques et dépendances nécessaires dès l'installation. À l'instar d'IDE populaires comme Microsoft Visual Studio ou NetBeans, Thonny vise à rendre le codage Python accessible et efficace, en particulier pour les débutants.



Logo de Thonny

3.2.2 Librairies

Cette partie détaille les bibliothèques logicielles exploitées pour faciliter le développement et enrichir les capacités du robot.

3.2.2.1 OpenCV

OpenCV est une bibliothèque open-source sous licence BSD, ce qui permet de l'utiliser librement et de l'intégrer dans d'autres projets. Sa popularité réside dans le fait qu'elle est utilisée dans de nombreux logiciels de traitement d'images et de vidéos. Développée et maintenue par une communauté de plus de 40 000 membres actifs, OpenCV offre plus de 2500 algorithmes de vision par ordinateur accessibles via des API. La vision par ordinateur, qui fait partie de l'intelligence artificielle,

vise à permettre aux machines d'analyser, de traiter et de comprendre des images, souvent capturées par des caméras. **OpenCV** permet de réaliser diverses tâches comme l'extraction de couleurs, la détection de visages, la reconnaissance de formes, et l'application de filtres. Ces algorithmes reposent principalement sur des calculs mathématiques, notamment ceux liés aux matrices, car une image peut être vue comme une matrice de pixels.[16].



Logo de OpenCV

3.2.2.2 SQLite3

Le module **SQLite3** de Python permet d'interagir facilement avec des bases de données **SQLite**, qui sont des bases de données légères, embarquées et stockées dans un simple fichier sur disque. Ce module est inclus par défaut avec Python depuis la version 2.5, ce qui signifie qu'il n'est pas nécessaire de l'installer séparément. Il fournit une interface conforme à la spécification DB-API 2.0, permettant d'exécuter des requêtes **SQL** pour créer, lire, mettre à jour ou supprimer des données. Grâce à sa simplicité et son intégration directe dans Python, **SQLite3** est une solution idéale pour les projets locaux, les applications légères ou les prototypes nécessitant une gestion efficace des données sans infrastructure serveur complexe.



Logo de SQLite

3.2.2.3 FlASK

Flask est un framework web léger et performant pour le langage Python. Souvent qualifié de « micro-framework », il fournit les fonctionnalités de base nécessaires au développement d'applications web, tout en restant simple et flexible. Contrairement à Django, qui intègre de nombreuses fonctionnalités comme l'authentification ou un panneau d'administration, Flask adopte une approche minimaliste : il laisse au développeur la liberté d'ajouter uniquement les composants dont il a réellement besoin [17].



Logo de Flask

3.2.2.4 PICAMERA2

Picamera2 est une bibliothèque Python développée pour faciliter l'utilisation des caméras du Raspberry Pi.Elle a pour but de faciliter la gestion des caméras connectées via le port dédié du Raspberry Pi tout en offrant un support limité pour certaines caméras USB. Fondé sur le projet open source **libcamera**, Picamera2 propose une interface plus simple et intuitive que les liaisons directes de libcamera, ce qui la rend parfaitement adaptée pour les projets en Python. Elle remplace l'ancienne bibliothèque PiCamera et propose des fonctionnalités similaires, tout en tirant pleinement parti du matériel d'imagerie du Raspberry Pi.

3.2.2.5 PYZBAR

Les codes-barres sont des représentations visuelles de données, conçues pour être lues automatiquement par des machines. En Python, on peut utiliser la bibliothèque **Pyzbar** pour créer un lecteur capable de décoder à la fois les codes-barres classiques et les codes QR [18].

Lorsqu'un code est détecté, **pyzbar** peut retourner trois types d'informations :

- **Type** : Le format du code détecté, comme `CODE-128`, `CODE-39` ou `QR-CODE`.
- **Données** : Le contenu du code (texte, chiffres, etc.), qui peut varier selon le type de code utilisé.

- **Emplacement** : Les coordonnées du code détecté dans l'image. Pour un code-barres, ce sont les bords du trait. Pour un QR code, ce sont les quatre coins du carré.

Conclusion

Ce chapitre nous a présenté les environnements matériels et logiciels nécessaires à la réalisation de notre robot assistant. Dans le chapitre suivant, nous détaillerons les différentes étapes de sa mise en œuvre : le câblage du circuit électrique, la programmation des modules, ainsi que des photos réelles illustrant l'assemblage et l'impression 3D des pièces mécaniques. Nous y aborderons également les limites rencontrées durant le développement du robot, ainsi que les perspectives d'amélioration et d'évolution possibles.

Chapitre 4

Réalisation, limites et perspectives

Plan

1 **Méthodologie d'intégration des composants** **31**

2 **Simulation du circuit électronique** **32**

3 **câblage du circuit** **33**

4 **Développement logiciel embarqué du robot assistant** **33**

5 **Limites du projet** **50**

6 **Perspectives d'amélioration** **51**

Introduction

Dans ce chapitre, nous abordons la conception et la réalisation du robot assistant humanoïde, en mettant l'accent sur l'assemblage matériel et la configuration logicielle nécessaires. Nous détaillerons le câblage des différents composants électroniques. De plus, nous présenterons la mise en place des modules logiciels, en particulier la gestion des bases de données SQLite pour l'enregistrement des utilisateurs via QR codes, ainsi que la programmation des fonctions spécifiques, Ce chapitre vise à fournir une description détaillée des étapes clés de la mise en œuvre du projet, de la configuration du matériel à l'intégration des fonctionnalités logicielles.

4.1 Méthodologie d'intégration des composants

L'intégration matérielle des différents éléments du robot assistant humanoïde repose sur une démarche rigoureuse visant à assurer la fiabilité du montage, la compatibilité électrique entre composants et une organisation claire facilitant le développement logiciel ultérieur.

Avant d'effectuer le câblage physique, une étude préalable des broches de tous les composants a été réalisée. Cette étape était essentielle pour comprendre le rôle de chaque pin (alimentation, masse, signal) et garantir un branchement cohérent avec les exigences électriques et logiques du système. Parmi les éléments étudiés figuraient notamment la carte Raspberry Pi 4, les capteurs environnementaux (DHT11, MQ-5, HC-SR04), ainsi que les modules d'affichage et les interfaces de commande.

Cette étude a permis de définir précisément les connexions nécessaires pour chaque composant, en tenant compte des spécificités de la carte Raspberry Pi, qui offre un nombre limité de broches GPIO. Des choix ont été faits afin d'optimiser l'utilisation de ces broches tout en assurant une compatibilité électrique entre les différents éléments.

Une attention particulière a été portée à l'organisation physique du câblage sur la plaque d'essai, dans le but de minimiser les interférences et de simplifier le repérage des connexions. La répartition des tensions (3.3V et 5V) a également été analysée afin d'éviter toute erreur susceptible d'endommager les composants sensibles. Des résistances de tirage ont été ajoutées lorsque nécessaire, notamment pour la ligne de données du capteur DHT11.

Enfin, chaque connexion a été testée et validée progressivement pour s'assurer du bon fonctionnement du système. Cette méthode d'intégration nous a permis de bâtir une base matérielle stable et fonctionnelle, sur laquelle les différentes briques logicielles ont ensuite pu être développées et déployées.

4.2 Simulation du circuit électronique

Pour illustrer le câblage de notre circuit électronique, nous avons utilisé le logiciel open source **Fritzing**. Cet outil open-source est particulièrement pratique pour créer des schémas électroniques clairs et visuels, il offre une bibliothèque riche en composants et permet de reproduire des schémas de montage à partir de composants électroniques réalistes, en reproduisant fidèlement une plaque d'essai (breadboard), un schéma électrique ou encore un circuit imprimé (PCB). À l'aide de Fritzing,, nous avons pu simuler notre montage de manière claire et compréhensible, ce qui facilite non seulement la phase de conception mais aussi celle de l'explication technique. L'interface facile à utiliser de Fritzing permet de positionner facilement les composants (comme les capteurs, microcontrôleurs ou moteurs) et de les relier avec des fils, comme sur un vrai montage. Cela nous a aidés à vérifier visuellement le câblage du circuit avant de le réaliser en pratique.

Logo de Fritzing

La figure suivante illustre la simulation du câblage du circuit électronique du robot, réalisée dans le but de valider visuellement l'intégration des composants avant le montage réel.

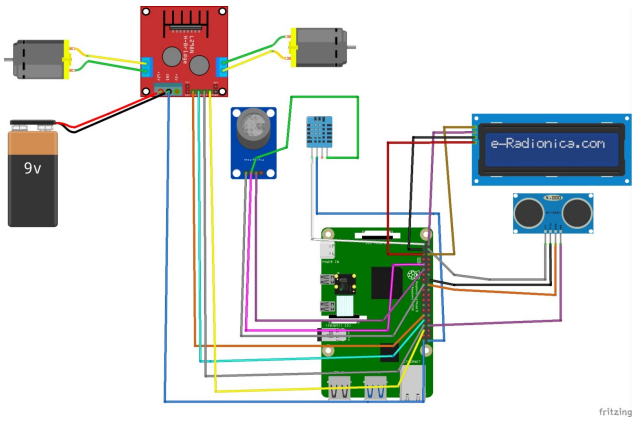


Figure 4.1 : Simulation du câblage du circuit du robot assistant

4.3 câblage du circuit

Voici une vue d'ensemble du câblage réel du robot, montrant la disposition des différents composants électroniques et capteurs utilisés pour le bon fonctionnement du système.

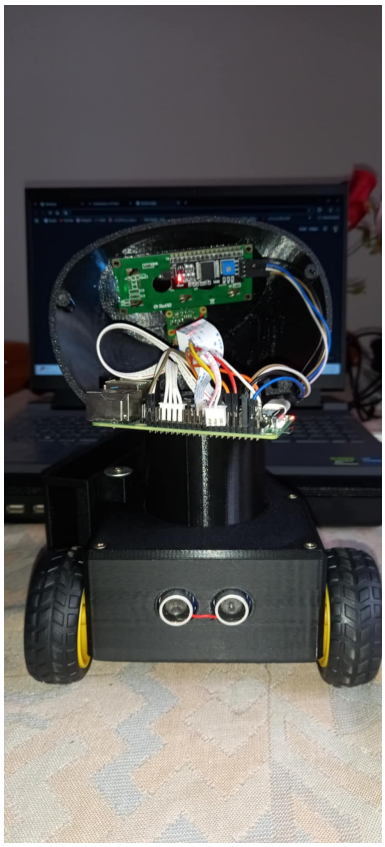


Figure 4.2 : Câblage réel du robot assistant humanoïde

4.4 Développement logiciel embarqué du robot assistant

La conception et la réalisation du robot assistant reposent en grande partie sur un développement logiciel embarqué rigoureux. Cette partie du projet vise à implémenter les fonctionnalités essentielles permettant au robot d'interagir avec son environnement, de traiter les données issues des capteurs, et de communiquer avec l'utilisateur de manière fluide et efficace. Dans cette section, nous détaillerons les différentes étapes, outils et méthodes utilisés pour développer le logiciel embarqué du robot assistant.

4.4.1 Programmation à distance du Raspberry Pi avec VS Code (Remote-SSH)

Dans ce projet, nous avons utilisé une méthode pratique pour créer et exécuter des fichiers à distance sur le Raspberry Pi en utilisant Visual Studio Code (VS Code) installé sur un ordinateur.

Pour cela, il suffit d'installer l'extension **Remote - SSH** dans VS Code. Cette extension permet d'établir une connexion SSH entre l'ordinateur et la carte Raspberry Pi. Une fois connectés, nous pouvons créer des fichiers, écrire du code et l'exécuter directement sur le Raspberry Pi, le tout depuis l'interface de VS Code. C'est une solution idéale lorsque la carte fonctionne en mode sans interface graphique (headless), car elle permet un développement fluide sans avoir besoin d'un écran, d'un clavier ou d'une souris connectés au Raspberry Pi.

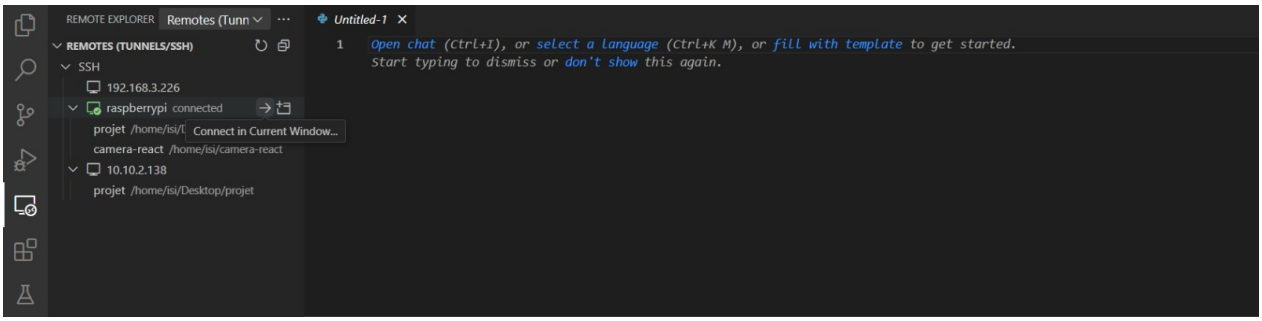


Figure 4.3 : Connexion SSH à distance entre VS Code et Raspberry Pi

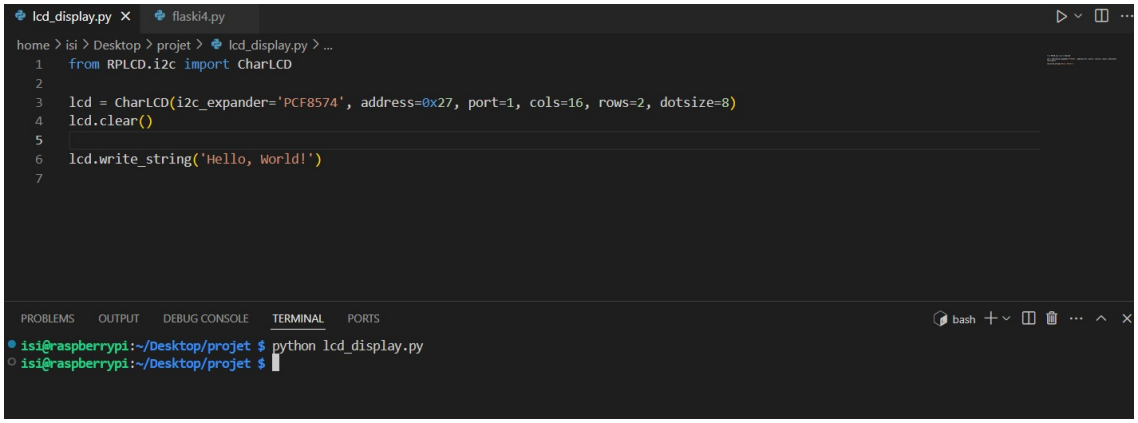


Figure 4.4 : Exécution d'un fichier Python sur le Raspberry Pi à distance à l'aide de VS Code (SSH)

4.4.2 Installation de la base de données SQLite sur le Raspberry Pi

Pour gérer localement des données sur le Raspberry Pi sans avoir recours à un serveur de base de données distant, nous avons opté pour l'utilisation de **SQLite**, une base de données légère, rapide et autonome. Son installation se fait très simplement via le terminal. Il suffit d'exécuter la commande suivante :

```
isi@raspberrypi:  $ sudo apt-get install sqlite3
```

Cette instruction télécharge et installe l'outil `sqlite3`, permettant de créer, interroger et manipuler des bases de données directement depuis le terminal. Grâce à SQLite, nous pouvons stocker des informations localement sur le Raspberry Pi de manière structurée, sans configuration complexe.

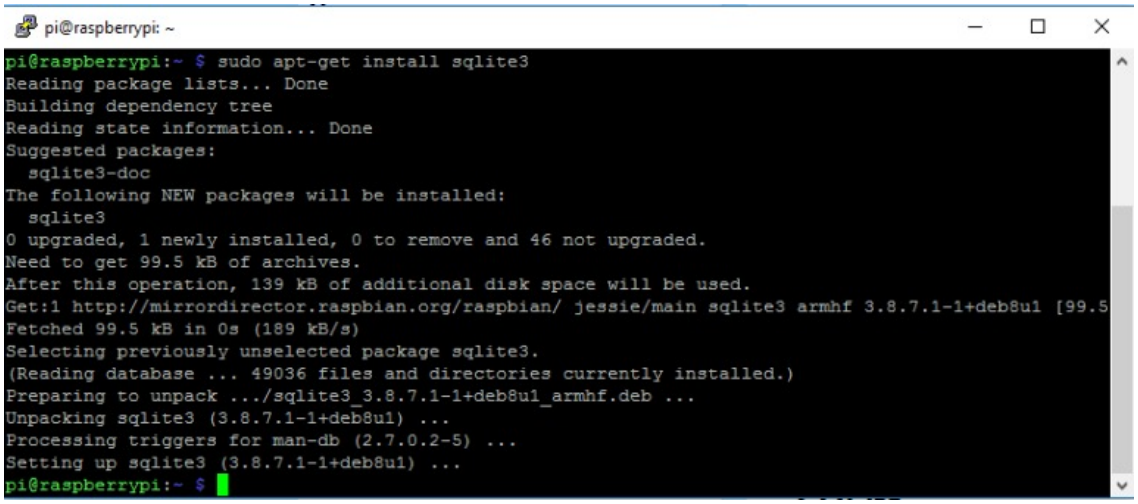


Figure 4.5 : Téléchargement et installation de SQLite3 via le terminal sur Raspberry Pi

Après l'installation de `sqlite3` sur le Raspberry Pi, une base de données locale est créée avec une table nommée `badges`. Cette table enregistre les utilisateurs identifiés par leurs codes QR. Elle est définie comme suit :

```
isi@raspberrypi:~ $ sqlite3 badges.db
SQLite version 3.40.1 2022-12-28 14:03:47
Enter ".help" for usage hints.
sqlite> CREATE TABLE badges(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    qr_code TEXT NOT NULL UNIQUE
);
```

Figure 4.6 : Code SQL de création de la table `badges` sous SQLite

Ensuite, nous insérons des données de test dans cette table à l'aide de la commande suivante :

```
isi@raspberrypi:~ $ sqlite3 badges.db
SQLite version 3.40.1 2022-12-28 14:03:47
Enter ".help" for usage hints.
sqlite> INSERT INTO badges (name, qr_code) VALUES
('Aymen Mansouri', 'QR-8171'),
('Dhia Soudani', 'QR-2003'),
('Mohamed Glaoui', 'QR-2025');
```

Figure 4.7 : Structure de la table `badges` dans la base de données SQLite

4.4.3 Lecture des QR codes et affichage via la base de données SQLite

Afin de permettre au robot d'identifier les utilisateurs à l'aide de leur code QR, nous avons développé un script Python intégrant plusieurs composants essentiels : la caméra, la bibliothèque de décodage des QR codes `pyzbar`, la base de données SQLite pour vérifier les utilisateurs enregistrés, ainsi qu'un écran LCD I2C pour afficher les noms reconnus. Ce script permet de capturer une image, détecter et décoder un QR code, interroger la base de données locale, et afficher le nom correspondant si l'utilisateur est reconnu.

4.4.3.1 Importation des bibliothèques nécessaires

Nous commençons par importer les bibliothèques nécessaires à l'exécution du programme, notamment pour la gestion de la caméra, le décodage de QR codes, l'affichage sur écran LCD, la manipulation de la base de données et la synchronisation des threads. Voici le rôle de chaque bibliothèque importée :

- **cv2** (OpenCV) : utilisée pour la capture et le traitement des images, notamment l'affichage de la vidéo et l'ajout de texte ou de formes sur les images.
- **sqlite3** : permet la gestion de la base de données locale, utilisée ici pour vérifier les utilisateurs à partir de leurs codes QR.
- **threading** : utilisée pour sécuriser l'accès concurrent à la base de données grâce à un verrou multithread.
- **pyzbar** : sert à décoder les QR codes capturés par la caméra.
- **picamera2** : permet de contrôler la caméra du Raspberry Pi pour capturer des images ou vidéos.

- **libcamera.Transform** : utilisée pour configurer l'orientation de l'image (ex. : miroir horizontal ou vertical).
- **RPLCD.i2c** : permet de piloter un écran LCD via l'interface I2C pour afficher des messages.
- **sleep** (de la bibliothèque **time**) : introduit des pauses temporaires dans l'exécution du programme (par exemple, pour laisser un message affiché quelques secondes).

4.4.3.2 Définition des paramètres d'affichage OpenCV

Cette partie du programme configure les paramètres nécessaires à l'affichage du texte sur les images capturées par la caméra. Ces paramètres incluent notamment la police d'écriture, la taille du texte, l'épaisseur des traits, ainsi que la couleur utilisée pour afficher le nom de l'utilisateur reconnu à l'écran.

4.4.3.3 Connexion à la base de données SQLite

Dans cette partie, une connexion est établie avec la base de données locale contenant les utilisateurs autorisés. Le paramètre `check_same_thread=False` est utilisé pour autoriser l'accès à la base depuis plusieurs threads, ce qui est essentiel dans un environnement multithreadé. Pour garantir la sécurité des accès concurrents, un mécanisme de verrouillage est également mis en place à l'aide de la bibliothèque `threading`.

4.4.3.4 Initialisation de l'écran LCD I2C et de la caméra

Au démarrage du système, l'écran LCD I2C est initialisé afin d'afficher un message de bienvenue signalant que le robot est prêt à fonctionner. Ce message est temporairement affiché, puis l'écran est effacé pour libérer l'affichage.

En parallèle, la caméra embarquée est configurée avec une résolution adaptée et une orientation correcte grâce à des paramètres de transformation. Cette configuration permet de capturer les images nécessaires, notamment pour la détection et la lecture des QR codes.

4.4.3.5 Fonction de vérification dans la base de données

Cette fonctionnalité permet de vérifier si une donnée extraite d'un QR Code correspond à un utilisateur enregistré dans la base de données. Lorsqu'un QR Code est détecté, une requête est effectuée dans la table `badges`. Si une correspondance est trouvée, le nom de l'utilisateur est récupéré ; sinon, le système retourne la mention « Inconnu ».

Dans un second temps, lorsque des QR Codes sont présents dans le champ de vision de la caméra, leurs coordonnées sont analysées afin d'afficher à l'écran, à proximité du code détecté, le nom de l'utilisateur associé. Cette information est affichée directement sur l'image capturée grâce à la bibliothèque `OpenCV`, en utilisant des fonctions de dessin de texte et de traitement d'image.

4.4.3.6 Fonction principale de détection des QR codes

Cette fonction constitue le cœur du système de reconnaissance par QR code. Elle assure une capture continue des images à partir de la caméra embarquée, et analyse chaque image pour détecter la présence éventuelle de QR codes. Lorsqu'un QR code est détecté, ses coordonnées sont récupérées, puis les données qu'il contient sont décodées.

Ces données sont ensuite comparées à celles enregistrées dans la base de données afin d'identifier l'utilisateur associé. Le nom correspondant est alors affiché à deux niveaux : visuellement sur l'image via un encadrement et une étiquette, et sur l'écran LCD du robot. Par ailleurs, les informations décodées sont également affichées dans la console pour faciliter le suivi du processus en temps réel.



Figure 4.8 : Affichage en temps réel des informations d'un QR code détecté

4.4.4 Présentation de l'application embarquée

Cette sous-section décrit le fonctionnement général du logiciel embarqué qui contrôle le robot assistant. Le système prend en charge la détection des conditions environnementales, la synthèse et la reconnaissance vocale, la communication avec différentes interfaces de programmation d'application (API) pour obtenir des données externes comme la météo, ainsi que la gestion des moteurs et de l'affichage sur l'écran LCD. De plus, il intègre une fonctionnalité de détection d'objets via la caméra.

L'ensemble du logiciel est organisé de manière à assurer le traitement simultané des différentes tâches essentielles, permettant ainsi une interaction réactive et naturelle avec l'utilisateur.

4.4.4.1 Importation des bibliothèques nécessaires

Les bibliothèques suivantes sont également importées et jouent un rôle essentiel dans le fonctionnement global du système :

- **os** et **json** : permettent respectivement la gestion des opérations système et le traitement des données au format JSON.
- **requests** : utilisée pour envoyer des requêtes HTTP, notamment pour communiquer avec des API externes.
- **speech_recognition** (importée sous le nom **sr**) : permet la reconnaissance vocale à partir de sources audio.
- **gTTS** : utilisée pour convertir du texte en parole (Text-to-Speech) en utilisant l'API Google Text-to-Speech.
- **numpy** (importée sous le nom **np**) : utilisée pour le traitement efficace de tableaux numériques, en complément de **cv2**.
- **flask**, **flask_socketio**, **flask_cors** : bibliothèques liées à l'interface web. **Flask** permet de créer le serveur web, **Flask-SocketIO** gère la communication en temps réel, et **Flask-CORS** autorise les requêtes croisées (Cross-Origin Resource Sharing).
- **time** : utilisée pour introduire des délais ou récupérer l'heure système.
- **board** et **adafruit_dht** : permettent la lecture des capteurs de température et d'humidité (DHT).
- **RPi.GPIO** : utilisée pour le contrôle des broches GPIO du Raspberry Pi.
- **random** : permet de générer des nombres aléatoires, par exemple pour la simulation ou la sélection aléatoire.

4.4.4.2 Initialisation du système

L'initialisation du système s'effectue en plusieurs étapes essentielles :

- **Initialisation de l'application web** : l'application web est mise en place à l'aide du framework **Flask**, enrichie par **Flask-SocketIO** pour la communication en temps réel, et **Flask-CORS** pour gérer les politiques de sécurité inter-domaines.

- **Chargement de la configuration** : les clés API et les paramètres du projet (comme la clé d'accès à Gemini ou à l'API météo) sont stockés dans un fichier `config.json` et chargés dynamiquement lors du démarrage de l'application.
- **Mise en place de l'écran LCD** : un écran LCD I2C (via la bibliothèque **RPLCD**) est initialisé pour permettre l'affichage local de messages simples à l'utilisateur.
- **Configuration des capteurs environnementaux** : un capteur DHT11 est initialisé pour lire la température et l'humidité ambiantes à l'aide de la bibliothèque **adafruit_dht**.
- **Configuration des broches GPIO** : les broches du Raspberry Pi sont configurées pour piloter les capteurs (gaz, ultrasons) et les moteurs. Le mode BCM est utilisé pour la numérotation des broches, et les broches sont préparées pour les entrées ou sorties selon leur usage.
- **Initialisation des moteurs** : les broches dédiées aux moteurs sont mises à l'état bas pour garantir que les moteurs restent à l'arrêt au démarrage.

L'ensemble de ces opérations permet de garantir un démarrage cohérent et sécurisé du système embarqué, assurant que tous les composants matériels et logiciels sont prêts à fonctionner de manière synchrone.

4.4.4.3 Intégration de l'intelligence artificielle avec l'API Gemini

L'assistant vocal repose sur une série de fonctions exploitant l'intelligence artificielle pour assurer une interaction fluide et intelligente en langage naturel entre l'utilisateur et le robot. Ces fonctions sont initialisées et prêtes à l'emploi dès le lancement du programme principal :

- **Fonction de synthèse vocale (`parler`)** : cette fonction transforme un texte en parole à l'aide de la bibliothèque **gTTS**. Le fichier audio généré est ensuite lu avec **mpg321**, permettant au robot de s'exprimer oralement.
- **Fonction de reconnaissance vocale (`ecouter`)** : elle capture la voix de l'utilisateur via le microphone (bibliothèque **speech_recognition**), ajuste le bruit de fond ambiant, et convertit l'audio en texte via l'API de Google. Elle est conçue pour gérer différents types d'erreurs (absence de voix, incompréhension, problème réseau).
- **Connexion à l'API Gemini (`envoyer_a_gemini`)** : cette fonction prend le texte reçu de l'utilisateur et l'envoie à l'IA Gemini via une requête POST. La réponse textuelle de l'IA est ensuite extraite, nettoyée et éventuellement raccourcie pour être lue vocalement ou affichée à l'utilisateur.



Logo de GEMINI

- **Journalisation des interactions** : chaque message envoyé ou reçu est sauvegardé localement dans un fichier `history.json`, permettant de conserver un historique des échanges vocaux. Cela facilite la traçabilité, l'analyse des réponses, et l'amélioration continue de l'assistant.

4.4.4.4 Contrôle des moteurs

Le robot se déplace grâce à deux moteurs commandés via des broches GPIO. Chaque fonction active les moteurs pour réaliser un mouvement spécifique :

- **Avancer** : les deux moteurs tournent vers l'avant.
- **Reculer** : les deux moteurs tournent en arrière.
- **Tourner à gauche** : un moteur avance pendant que l'autre recule.
- **Tourner à droite** : inverse du mouvement précédent.
- **Arrêt** : tous les moteurs sont coupés.

4.4.4.5 Capteurs environnementaux

Le robot utilise plusieurs capteurs pour surveiller son environnement. Les données collectées sont ensuite affichées sur l'écran LCD. Voici les principales fonctions :

- **Température et humidité (DHT11)** : mesure la température de l'air et le taux d'humidité. Si une erreur est détectée, des valeurs nulles sont retournées.
- **Capteur de gaz (MQ2)** : détecte la présence de gaz. Si l'air est propre, un message rassurant est affiché ; sinon, un avertissement s'affiche avec une estimation du niveau de gaz en ppm.
- **Capteur ultrason (HC-SR04)** : mesure la distance à un obstacle. Il utilise un signal sonore pour estimer cette distance en centimètres.
- **Météo externe (API OpenWeather)** : récupère la température extérieure et la description du temps (ex. : "nuageux") via Internet.
- **Affichage LCD** : les données de tous ces capteurs sont affichées une par une sur l'écran LCD, avec une pause entre chaque message pour bien les lire.

4.4.4.6 Fonctionnement général

Le système est basé sur deux boucles principales et une fonction additionnelle :

Boucle de l'assistant :

- Écoute et interprétation des commandes vocales de l'utilisateur (déplacement, arrêt, questions simples).
- Exécution d'actions physiques (comme se déplacer ou s'arrêter).
- Réponses vocales générées via l'API Gemini.

Boucle des capteurs environnementaux :

- Lecture continue des données des capteurs :
  - Température
  - Humidité
  - Gaz
  - Distance
  - Météo externe
- Affichage des données sur l'écran LCD.

Détection d'objets :

- Utilisation de la caméra pour identifier des objets présents dans le champ de vision.
- Affichage de cadres et d'étiquettes informatives grâce à un modèle pré-entraîné.

Apprentissage réalisé :

- La fonction `boucle_assistant()` constitue le cœur de l'intelligence vocale du robot. Elle permet la reconnaissance continue des commandes vocales, leur traitement contextuel (déplacement, arrêt, réponses) et l'exécution des actions associées. Une structure conditionnelle a été mise en place pour interpréter différentes formulations de commandes et déclencher les fonctions de déplacement ou de réaction. En parallèle, une connexion à l'API Gemini a été intégrée pour répondre aux questions ouvertes.
- Un apprentissage spécifique a également été réalisé en intégrant des réponses personnalisées concernant l'Institut Supérieur d'Informatique de Tunis, permettant au robot de fournir des réponses vocales précises sur l'établissement, ses formations ou son encadrement. D'autres questions récurrentes ont été anticipées et codées manuellement avec leurs réponses correspondantes, renforçant l'illusion d'un assista nt vocal intelligent et informé sur son environnement.

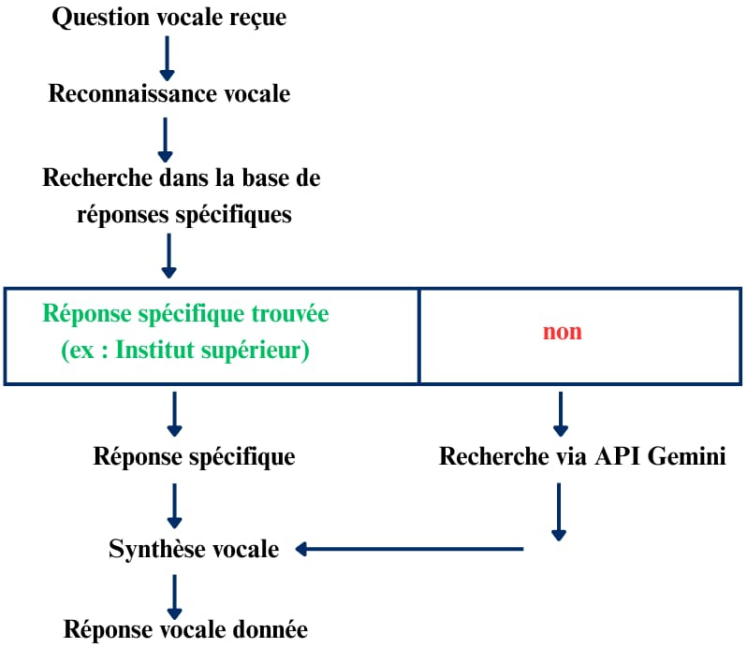


Figure 4.9 : Schéma de traitement d'une question vocale par le robot assistant

4.4.4.7 Routes de l'application Flask

L'application web repose sur plusieurs routes définies à l'aide du framework Flask. Ces routes permettent d'assurer l'interaction entre l'utilisateur et le robot via une interface web.

- `/` : route principale qui affiche l'interface web `index.html`.
- `/video` : diffuse en continu les images de la caméra du robot, avec détection d'objets intégrée, au format `MJPEG`.
- `/alerts` : retourne les dernières mesures environnementales (température, humidité, gaz, distance, météo) sous forme de réponse JSON.
- `/move` : reçoit des commandes de déplacement via une requête POST (avancer, reculer, tourner, arrêt) et exécute l'action correspondante sur le robot.
- `/chat` : retourne la dernière réponse générée par l'assistant vocal intelligent.
- `/download_chat` : permet de télécharger l'historique des échanges avec l'assistant sous forme d'un fichier texte.

4.4.4.8 Lancement principal de l'application

Le programme principal coordonne le lancement de toutes les fonctionnalités du robot :

- Deux threads sont initialisés pour exécuter en parallèle la boucle de l'assistant vocal et celle des capteurs environnementaux.
- Le serveur Flask est ensuite lancé avec `SocketIO` pour permettre une communication en temps réel via l'interface web.
- En cas d'interruption manuelle (par exemple, avec `Ctrl+C`), le programme effectue un nettoyage complet : arrêt de la caméra, libération du capteur DHT11, réinitialisation des broches GPIO, et effacement de l'écran LCD.

4.4.5 Interface Web de contrôle et de visualisation

Afin de faciliter l'interaction avec le robot assistant et de centraliser les informations collectées par les capteurs, une interface web a été développée. Cette interface, hébergée localement sur le Raspberry Pi via un serveur Flask, permet à l'utilisateur de visualiser en temps réel les données environnementales (température, humidité, gaz, etc.), de piloter les mouvements du robot à distance à l'aide de boutons de commande, et de consulter l'historique des conversations générées par l'assistant vocal. Accessible via un simple navigateur web, elle offre une solution intuitive, ergonomique et multiplateforme pour superviser le fonctionnement du robot.

4.4.5.1 Dashboard des capteurs

La section « Dashboard Capteurs » de la page web permet d'afficher en temps réel les données environnementales mesurées par le robot. Ces informations sont récupérées via des requêtes périodiques à l'API Flask embarquée dans le Raspberry Pi.

Le tableau de bord présente notamment :

- La température intérieure mesurée par le capteur DHT11,
- Le taux d'humidité relative,
- La détection de gaz (CO) avec un indicateur visuel «RAS» ou «Détecté»,
- La distance d'obstacles détectée à l'avant du robot,
- La température extérieure et les conditions météo récupérées via une API externe.

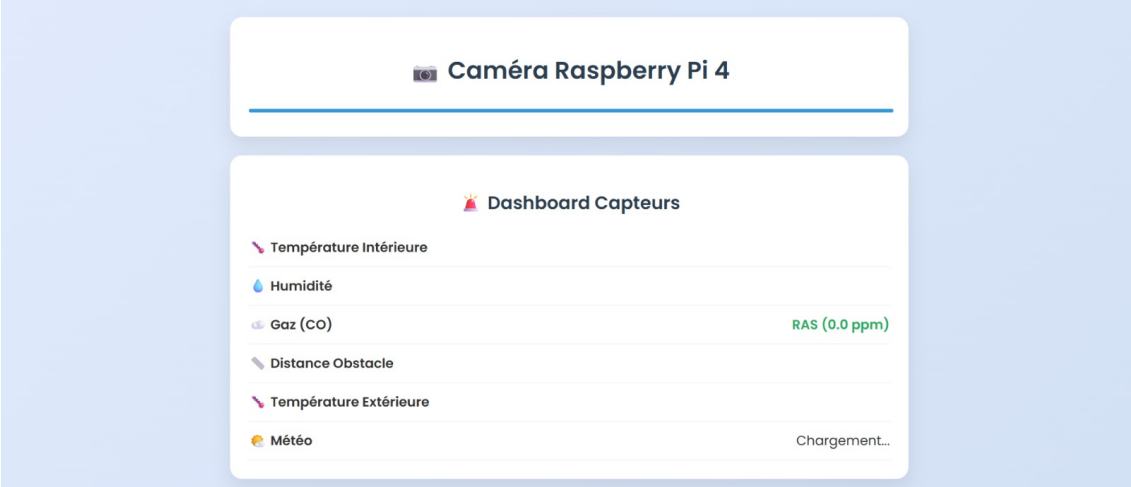


FIGURE 4.10 : Interface du dashboard des capteurs sur la page web

4.4.5.2 Contrôle du robot et affichage des réponses de l'assistant

Cette section de la page web offre deux fonctionnalités principales :

- **Télécommande du robot** : un ensemble de boutons permet à l'utilisateur de contrôler manuellement les déplacements du robot à distance. Les actions possibles sont : *Avancer*, *Reculer*, *Tourner à gauche*, *Tourner à droite* et *Stop*.
- **Dernier message de l'assistant vocal** : cette zone affiche la dernière réponse générée par l'API Gemini à la suite d'une commande vocale détectée par le robot. Elle permet de garder une trace rapide et visuelle des échanges récents.

Un bouton supplémentaire permet également de télécharger l'historique complet de la conversation en local.

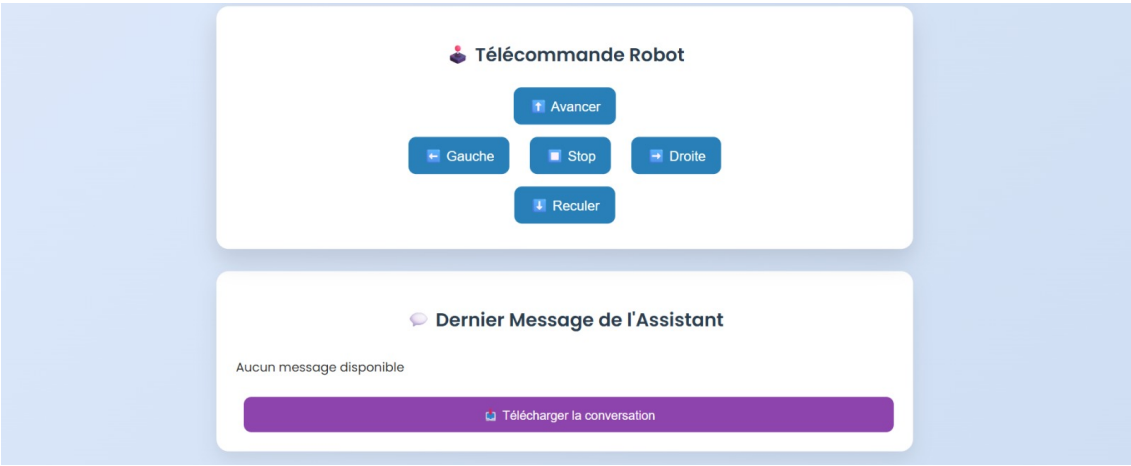


FIGURE 4.11 : Interface de contrôle du robot et affichage du dernier message de l'assistant

4.4.5.3 Historique des conversations

L'interface permet également la génération automatique d'un historique des échanges entre l'utilisateur et l'assistant vocal. Chaque session est sauvegardée dans un fichier texte nommé sous la forme `chat(n).txt`, où `n` représente l'index de la session.

Ces fichiers sont hébergés localement sur le serveur Flask intégré au Raspberry Pi, et sont accessibles via l'adresse IP du robot, typiquement : `http://192.168.1.18:5000`.

L'utilisateur peut consulter ces fichiers, les télécharger ou les afficher directement dans le dossier correspondant via un simple clic sur le bouton prévu à cet effet.

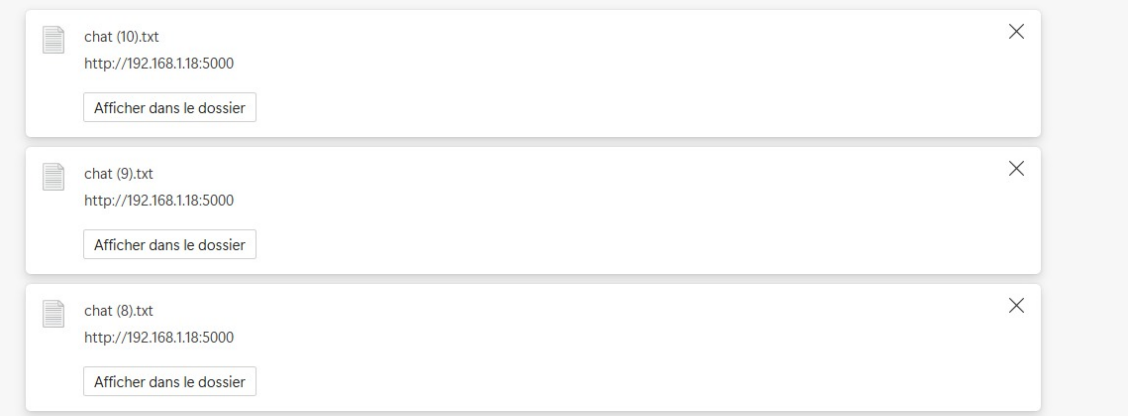


Figure 4.12 : Exemple de fichiers d'historique des conversations générés automatiquement

4.4.6 L'emboîtement du robot assistant

Cette section présente la manière dont les différentes pièces du robot assistant s'imbriquent entre elles pour assurer une structure solide et fonctionnelle.

4.4.6.1 Conception de l'emboîtement du robot assistant

Pour protéger les pièces électroniques et assurer une intégration esthétique et fonctionnelle du systéme, une conception d'un boîtier a été créée. La conception du robot assistant a été réalisée avec le logiciel Fusion 360, permettant la fabrication de modèles 3D précis prêts à l'impression ou à la production. Cette étape a permis l'établissement de la disposition interne des capteurs, de la carte Raspberry Pi, de l'écran LCD ainsi que de l'alimentation, en considérant des facteurs comme l'espace, de ventilation et la facilité d'accès.

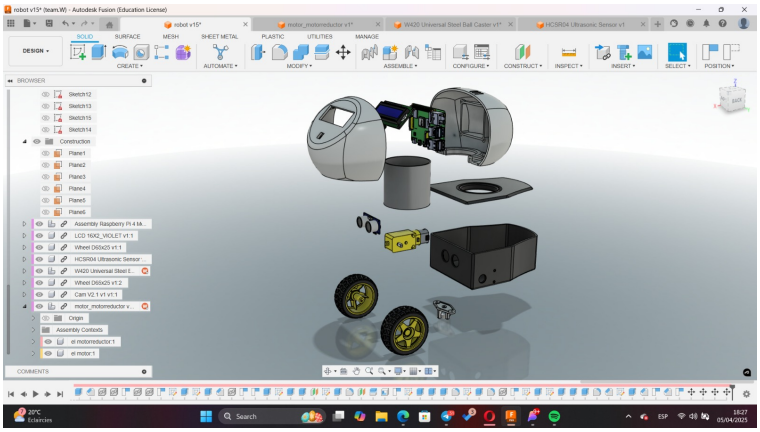


Figure 4.13 : Modélisation 3D de l'emboîtement du robot assistant (vue éclatée) sous Fusion 360

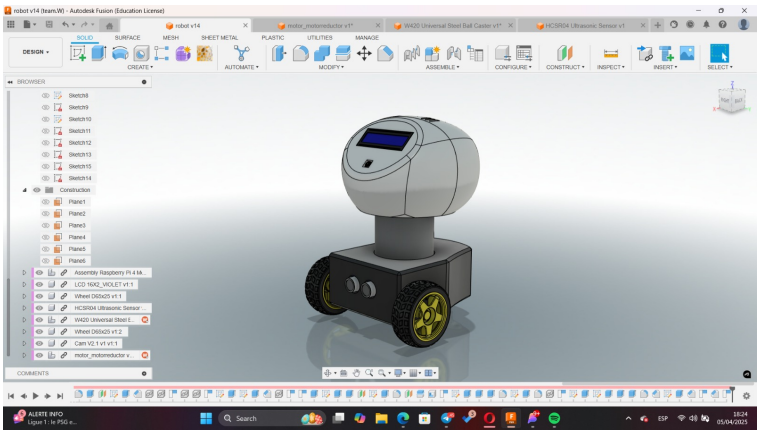


Figure 4.14 : Modélisation 3D de l'emboîtement du robot assistant (vue assemblée) sous Fusion 360

4.4.6.2 Impression 3D de l'emboîtement

Une fois la modélisation terminée, l'emboîtement a été exporté au format compatible avec l'imprimante 3D. L'impression a ensuite été effectuée à l'aide de l'imprimante 3D disponible au sein du centre 4C. Cette opération nous a permis d'obtenir un boîtier physique solide, adapté aux dimensions des composants, et facilitant leur intégration de manière stable et sécurisée.



Figure 4.15 : Imprimante 3D utilisée au sein du centre 4C

La **Figure 4.16** montre la phase d'impression 3D d'une pièce d'emboîtement du robot. Cette étape est cruciale pour assembler solidement les différentes parties de la structure du robot, tout en respectant les dimensions spécifiques du design. L'impression 3D permet une grande flexibilité dans la conception des pièces mécaniques, facilitant ainsi l'intégration des capteurs, des composants électroniques et des éléments structurels du robot :

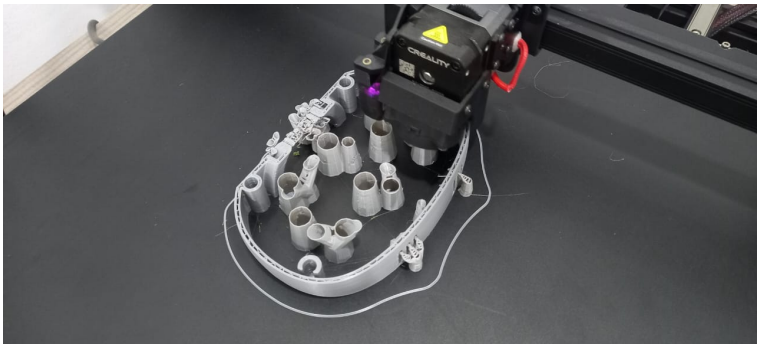


Figure 4.16 : Phase d'impression 3D de l'emboîtement du robot

La **Figure 4.17** présente les composants du robot une fois l'impression 3D terminée. Ces pièces, conçues sur mesure, ont été fabriquées pour assurer un assemblage précis et une intégration optimale des différents modules électroniques et mécaniques. L'utilisation de l'impression 3D a permis de produire des formes complexes adaptées aux besoins spécifiques du projet, tout en réduisant les coûts et les délais de fabrication :



Figure 4.17 : Composants imprimés en 3D après la fabrication

La **Figure 4.18** présente le robot humanoïde dans son état final, après l'assemblage complet de l'ensemble des composants mécaniques imprimés en 3D.



Figure 4.18 : Robot humanoïde final assemblé avec les composants imprimés en 3D

Les deux figures ci-dessous présentent respectivement la vue latérale gauche et la vue latérale droite du robot assistant humanoïde. Bien que les capteurs ne soient pas directement visibles sur ces images, ces vues permettent d'illustrer la forme générale du robot, la disposition de sa structure externe ainsi que l'emplacement approximatif des composants embarqués. Elles offrent un aperçu global du design mécanique du robot et de son intégration physique dans l'environnement.



Figure 4.19 : Vue latérale gauche



Figure 4.20 : Vue latérale droite

4.5 Limites du projet

Cette section met en lumière les principales limites rencontrées lors de la conception et du développement du robot, afin de mieux comprendre les contraintes qui ont orienté le projet.

4.5.1 Fonctionnalités non implémentées

Certaines fonctionnalités prévues n'ont pas pu être intégrées dans ce projet, notamment en raison de contraintes techniques et temporelles. Le robot ne dispose pas encore d'une adaptation linguistique au-delà du français, limitant ainsi son usage à un public francophone. De plus, la

reconnaissance et la gestion d'obstacles en temps réel ne sont pas encore développées, restreignant son utilisation à des environnements intérieurs stables et dégagés. Enfin, les interactions vocales restent basiques et pourraient être enrichies pour offrir une expérience utilisateur plus complète.

4.5.2 Contraintes techniques et environnementales

Conçu pour évoluer principalement en intérieur, le robot doit opérer dans des espaces aux conditions relativement contrôlées. Il ne supporte pas les terrains irréguliers ni les obstacles mobiles complexes, ce qui limite son champ d'action. Par ailleurs, son autonomie est limitée par une alimentation par batterie, nécessitant une gestion régulière de la charge. La connexion Internet stable est également indispensable pour les fonctions en ligne telles que l'affichage météo en temps réel ou les réponses vocales via API ; toute instabilité réseau impacte directement la qualité du service. Enfin, le robot n'a pas été conçu pour résister à des conditions environnementales extrêmes (forte humidité, poussière, variations de température), ce qui limite son usage dans certains contextes spécifiques.

4.6 Perspectives d'amélioration

Plusieurs axes d'amélioration sont envisageables pour enrichir les capacités du robot et dépasser ses limites actuelles. Tout d'abord, l'intégration d'une gestion plus avancée des obstacles, grâce à des capteurs supplémentaires et des algorithmes de navigation adaptative, permettrait d'élargir son domaine d'utilisation à des espaces plus complexes. L'extension du support linguistique offrirait également une plus grande accessibilité à un public international.

Sur le plan technique, optimiser la consommation énergétique ou intégrer des solutions de recharge automatique pourrait prolonger son autonomie et améliorer son autonomie opérationnelle. Par ailleurs, le développement d'interactions vocales plus riches et personnalisées, couplées à une intelligence artificielle embarquée plus performante, rendrait le robot plus réactif et agréable à utiliser. Enfin, le renforcement de la robustesse environnementale permettrait d'envisager des usages dans des contextes variés, allant au-delà du simple intérieur stable.

Conclusion

Ce chapitre a présenté l'emboîtement mécanique du robot assistant ainsi que les étapes de fabrication par impression 3D, garantissant une structure stable et adaptée aux composants électroniques. Les contraintes techniques et les limites fonctionnelles ont été mises en évidence, tout comme les axes d'amélioration envisageables pour enrichir les capacités du robot. Ces éléments constituent une base essentielle pour les développements futurs du projet.

Conclusion générale

Le développement de la robotique et de l'intelligence artificielle a pris une place croissante dans de nombreux domaines à travers le monde, notamment dans les secteurs de l'assistance, de la santé, de l'éducation et de la sécurité. En Tunisie, bien que la robotique soit encore limitée, ce projet de fin d'études a permis d'apporter une solution innovante visant à répondre aux besoins locaux en matière d'assistance et de services intelligents.

Le travail réalisé nous a permis de concevoir et de réaliser un robot assistant humanoïde, capable d'effectuer des tâches quotidiennes simples, telles que la gestion des capteurs environnementaux, la communication via une API, et l'interaction avec l'utilisateur au travers d'une interface vocalisée. Le robot présente un modèle de solution qui, malgré les défis technologiques rencontrés, ouvre la voie à un usage plus large de la robotique dans le milieu académique et au-delà.

Le projet a atteint ses principaux objectifs : concevoir un robot fonctionnel répondant à des critères spécifiques, intégrer des technologies variées telles que la reconnaissance vocale et l'affichage LCD, et offrir une réponse à une problématique locale d'assistance et d'optimisation des tâches courantes. Toutefois, certains objectifs restent partiellement atteints, notamment en ce qui concerne l'optimisation des performances du robot dans des environnements non contrôlés, ce qui représente un axe d'amélioration pour de futures évolutions.

Sur le plan professionnel, cette expérience nous a permis de renforcer nos compétences techniques, notamment dans la programmation embarquée, la conception de systèmes robotiques et l'intégration de diverses technologies. Cette expérience nous a aussi permis de mieux comprendre les défis d'un projet en milieu académique, notamment la gestion du temps et des ressources limitées.

D'un point de vue personnel, ce projet nous a enrichis sur le plan humain en nous confrontant à des situations complexes et en nous permettant de travailler de manière collaborative. Il nous a également permis de mieux comprendre l'impact de la robotique dans notre société et d'appréhender les défis technologiques auxquels nous sommes confrontés.

Enfin, bien que ce projet ait été une étape importante dans la réalisation de notre parcours, il reste des perspectives de développement, notamment dans l'amélioration de l'autonomie du robot, l'adaptation à d'autres environnements, et l'introduction de nouvelles fonctionnalités pour enrichir l'interaction utilisateur.

Bibliographie

[1] Wikipédia. « Robot. » [Consulté le 4 fév mai 2025]. (fév 2025), adresse : https://fr.wikipedia.org/wiki/Robot.

[2] Wikipédia. « Robot. » [Consulté le 4 fév 2025]. (Fév 2025), adresse : [https://www.robot-magazine.fr/robot-humanoide/].

[3] Génération Robots. « À la découverte des robots humanoïdes : un regard approfondi sur leurs applications. » [Consulté le 4 fév 2025]. (2025), adresse : https://www.generationrobots.com/blog/fr/a-la-decouverte-des-robots-humanoides-un-regard-approfondi-sur-leurs-applications/.

[4] Aldebaran Robotics. « Pepper, le robot humanoïde. » [Consulté le 4 fév 2025]. (2025), adresse : https://aldebaran.com/fr/pepper/.

[5] Wikipédia. « NAO (robotique). » [Consulté le 4 fév 2025]. (2025), adresse : https://fr.wikipedia.org/wiki/NAO_(robotique).

[6] Wikipédia. « ASIMO. » [Consulté le 4 fév 2025]. (2025), adresse : https://fr.wikipedia.org/wiki/ASIMO.

[7] Wikipédia. « Atlas (robot). » [Consulté le 4 fév 2025]. (2025), adresse : https://fr.wikipedia.org/wiki/Atlas_(robot).

[8] Wikipédia. « Sophia (robot). » [Consulté le 4 fév 2025]. (2025), adresse : https://fr.wikipedia.org/wiki/Sophia_(robot).

[9] Ignition Program. « La méthode Scrum pour les nuls. » [Consulté le 15 fév 2025]. (2025), adresse : https://ignition-program.com/tuto/la-methode-scrum-pour-les-nuls.

[10] Lucidchart. « Langage UML : comprendre la modélisation UML. » [Consulté le 27 fév 2025]. (2025), adresse : https://www.lucidchart.com/pages/fr/langage-uml.

[11] IBM Documentation. « Diagrammes de cas d'utilisation. » [Consulté le 27 fév 2025]. (2025), adresse : https://www.ibm.com/docs/fr/dmrt/9.5.0?topic=diagrams-use-case.

[12] Lucidchart. « Diagramme de séquence UML : exemple, explication et tutoriel. » [Consulté le 27 fév 2025]. (2025), adresse : https://www.lucidchart.com/pages/fr/diagramme-de-sequence-uml.

[13] Lucidchart. « Diagramme de classes UML : explication et exemples. » [Consulté le 27 fév 2025]. (2025), adresse : https://www.lucidchart.com/pages/fr/diagramme-de-classes-uml.

[14] Saagie. « Visual Studio Code. » [Consulté le 23 mars 2025]. (2025), adresse : https://www.saagie.com/fr/blog/technologie/visual-studio-code/.

[15] RealVNC. « Qu'est-ce que la technologie d'accès à distance VNC ? » [Consulté le 23 mars 2025]. (2025), adresse : https://discover.realvnc.com/what-is-vnc-remote-access-technology.

[16] Axopen. « OpenCV, c'est quoi ? » [Consulté le 27 mars 2025]. (2025), adresse : https://www.axopen.com/blog/2019/09/open-cv-cest-quoi/.

[17] GeeksforGeeks. « Flask Tutorial. » [Consulté le 27 mars 2025]. (2025), adresse : https://www.geeksforgeeks.org/flask-tutorial/.

[18] GeeksforGeeks. « How to Make a Barcode Reader in Python ? » [Consulté le 28 mars 2025]. (2024), adresse : https://www.geeksforgeeks.org/how-to-make-a-barcode-reader-in-python/.

Annexes

Vous pouvez consulter tous les codes sources utilisés dans ce projet via le lien suivant :

`https://here-project.netlify.app/`

ملخّص

في إطار تربص التخرج بالمعهد العالي للإعلامية، تم إنجاز هذا المشروع للحصول على الإجازة الوطنية في الإلكترونيك، الكهروتقنية والآلية. يهدف هذا العمل إلى تصميم وإنجاز روبوت مساعد بشري قادر على التفاعل مع محيطه وتقديم المساعدة في الأنشطة اليومية داخل المؤسسة. يدمج الروبوت مستشعرات بيئية، كاميرا للتعرف على رموز صض، مساعداً صوتياًا ذكياًا يعتمد على واجهات برمجة التطبيقات، بالإضافة إلى واجهة ويب للتحكم عن بُعد. يعزز النظام المقترح التفاعل السلس مع المستخدم، عرض البيانات في الوقت الحقيقي، والتنقل الذاتي بفضل التحكم في المحركات وكشف العوائق.

كلمات مفاتيح : روبوت بشري، راسبيري باي، مساعد صوتي، رمز صض، إنترنت الأشياء، واجهة ويب، مراقبة بيئية.

Résumé

Dans le cadre de notre stage de fin d'études à [Nom de l'entreprise ou laboratoire], ce projet a été réalisé pour l'obtention du diplôme national de licence en Électronique, Électrotechnique et Automatique. Il a pour objectif de concevoir et développer un robot assistant humanoïde capable d'interagir avec son environnement et d'assister les utilisateurs dans des tâches quotidiennes. La solution proposée repose sur une plateforme robotique équipée de capteurs et d'actionneurs, d'un module de reconnaissance vocale, ainsi que d'une interface utilisateur permettant un contrôle et un suivi à distance via une application mobile ou web.

Mots clés : Robot humanoïde, Raspberry Pi, assistant vocal, QR code, interaction homme-machine, IoT.

Abstract

As part of our final internship at [Company or Laboratory Name], we have carried out this work in order to obtain the national license diploma in Electronics, Electrotechnics and Automation. The objective of this project is to design and develop a humanoid assistant robot capable of interacting with its environment and assisting users in daily tasks. The proposed solution includes a robotic platform equipped with sensors and actuators, a voice recognition module, and a user interface allowing remote control and monitoring via a mobile or web application.

Keywords : Humanoid Robot, Raspberry Pi, Voice Assistant, QR Code, Human-Machine Interaction, IoT.

نهج بحيرة ملاران ـ ضفاف البحيرة ـ تونس الهاتف : 111 111 71 الفاكس : 222 222 71 البريد الالكتروني : contact@company.com
Rue du Lac Malaren, Les Berges du Lac 1053 Tunis **Tél** : 71 111 111 **Fax** : 71 222 222 **Email** : contact@company.com

2 نهج أبو الريحان البارونى 2080 أريانة الهاتف : 164 706 71 الفاكس : 698 706 71 البريد الالكتروني : isi@isim.rnu.tn
2, Abou Raihane Bayrouni 2080 l'Ariana **Tél** : 71 706 164 **Fax** : 71 706 698 **Email** : isi@isim.rnu.tn